

Bi-Objective Ant Colony Optimization for Trajectory Planning and Task Offloading in UAV-Assisted MEC Systems

Yiqian Wang¹, Jie Zhu¹, Haiping Huang¹, *Member, IEEE*, and Fu Xiao¹, *Member, IEEE*

Abstract—In the paper, the Unmanned Aerial Vehicle (UAV) path planning and task offloading problem in UAV-assisted mobile edge computing (MEC) systems is investigated. A bi-criterion ant colony optimization (bi-ACO) framework is proposed for the considered problem with the objectives of minimizing the total cost and the completion time, meanwhile satisfying the energy, deadline, location, and priority constraints. In the bi-ACO framework, multiple heterogeneous colonies are introduced with different preferences of objectives. Each colony maintains five pairs of pheromone matrices for constructing feasible solutions. Besides the colony settings, three key components of bi-ACO are delicately designed: feasible solution generation method (FSGM) to construct a feasible solution, solution division method (SDM) to improve obtained solutions of good quality, and pheromone update method (PUM) to update pheromone matrices by pheromone evaporation operation and pheromone enhancement operation based on the preferences of colonies. Four Pareto-based metrics are introduced to evaluate the performance of the compared algorithms. Experimental results show that the proposal outperforms the compared baseline algorithms in effectiveness and robustness.

Index Terms—ACO, mobile edge computing, task offloading, trajectory planning, UAV.

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs) are gaining more and more popularity in a wide range of applications in transportation, disaster management, agriculture and health care.

Manuscript received 13 December 2023; revised 23 May 2024; accepted 28 May 2024. Date of publication 3 June 2024; date of current version 5 November 2024. This work was supported in part by Jiangsu Province Universities Natural Science Research Major Project under Grant 23KJA520010, in part by the State Key Laboratory of Computer Architecture (ICT,CAS) under Grant CARCHA202107, in part by the National Science Foundation for Postdoctoral Scientists of China under Grant 2018M640510, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20201375, and in part by the National Natural Science Foundations of China under Grant 62072252. Recommended for acceptance by B. Liang. (Yiqian Wang and Jie Zhu are co-first authors.) (Corresponding author: Haiping Huang.)

Yiqian Wang is with the Nanjing University of Posts and Telecommunications, Nanjing 210013, China.

Jie Zhu is with the Nanjing University of Posts and Telecommunications, Nanjing 210013, China, and also with the State Key Laboratory Chinese of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100864, China.

Haiping Huang and Fu Xiao are with the Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks, Nanjing University of Posts and Telecommunications, Nanjing 210013, China (e-mail: hhp@njupt.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2024.3408603>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2024.3408603

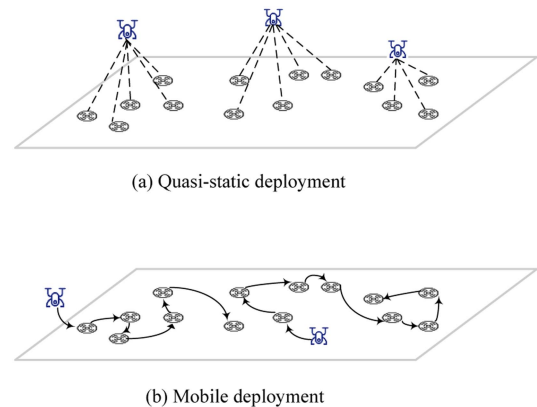


Fig. 1. UAV deployment modes.

Various difficulties in ground operations such as traffic congestion, extensive labor costs and severe weather conditions can be relieved by employing UAVs. In these UAV-assisted systems, UAVs can be provisioned as mobile base stations [1], mobile relays [2], mobile caches [3] and mobile surveillance [4].

One of the prevalent UAV-assisted innovative solutions is the UAV-assisted mobile edge computing (MEC) system where UAVs are equipped with powerful CPUs and taken as the roaming edge computing devices. In the general MEC systems, many smart devices (SDs) periodically collect and upload data to nearby base stations which then perform complex computing on these data and return results to SDs. The energy consumption of SDs is one of the major issues for the continuous availability of MEC applications. Introducing UAVs as mobile base stations can greatly reduce the energy consumption for data transmission which is the major energy consumption of SDs. Besides, the UAVs can also enhance the intelligent capabilities of MEC systems by playing as edge computing devices to perform complex computing on data.

In many UAV-assisted systems, UAVs are deployed as the air interfaces in the quasi-static manner [5], [6] (as shown in Fig. 1(a)). In these cases, each UAV serves for multiple SDs in a given area. The quasi-static UAVs may periodically adjust the locations but most of the time they stay at the fixed locations. If SDs are deployed sparsely, more quasi-static UAVs would be provisioned or the serving area of each UAV would be expanded. The former leads to a higher cost, and the latter reduces the quality of service.

The quasi-static deployment of UAVs does not take full advantage of the excellent maneuverability of the UAVs. Many benefits can be introduced by employing mobile UAVs in the UAV-assisted MEC systems (as shown in Fig. 1(b)). First, provisioning UAVs in the mobile manner is suitable for either sparse or dense deployment of SDs. Second, the locations of UAVs can be adaptive to the distribution of workload. Third, the UAVs can process data while flying to further improve the effectiveness of the systems.

In the paper, we consider an UAV scheduling problem where the trajectory planning and the multi-stage task offloading are jointly optimized. Many SDs are distributed in a given area and submit computation tasks periodically. The computation tasks are deadline-constraint and divided into three stages: the data transmission stage, the task computation stage and the result transmission stage. A fixed number of reserved UAVs are provisioned in several base stations and serving the area. In order to satisfy the deadline constraint in cases of heavy workload, an unlimited number of on-demand UAVs at a higher cost can be employed. The objectives are to minimize the total cost and the completion time of all tasks. The novelty of the problem under study lies in below. (i) The computation tasks are multi-stage tasks. Different task stages have different limitations for UAVs to fly or hover. (ii) Each SD should be visited twice for data fetching and result returning, and the time gap between two successive visits of a SD should be long enough to complete a task. (iii) UAVs are elastic resources with different price structures.

It is known that the multiple UAV trajectory planning problem has already proved to be NP-hard [7]. Therefore, it follows that the problem optimizing both the trajectory planning and the task offloading is also NP-hard. Offloading multi-stage tasks to geographically distributed and flying UAVs results in great challenges: (i) Energy management is critical for the considered problem. The energy capacity of each UAV is restricted. In order to gain an energy-efficient solution, the hovering time, the flying time and the time to change batteries should be elaborately planned. (ii) The trajectories of UAVs are more difficult to plan since each SD could be visited once or twice. An UAV can hover near a SD for the entire duration of the task, or leave during the computation stage and fly back for the result transmission. Therefore, a timetable of processing different stages of tasks should also be given. (iii) It is not easy to obtain an efficient task offloading solution. The sequence of SDs to visit in a trajectory planning only determines the order of the data and result transmission stages of tasks. The order of the computation stages of tasks assigned to the same UAV should be delicately scheduled. (iv) It is difficult to balance the conflicting objectives. In order to reduce the total cost, it is better to employ the on-demand UAVs as less as possible. However, the reserved UAVs may be overloaded and the deadline constraint may be not guaranteed or the completion time may be extended.

The paper aims to provide an effective and efficient method for the multiple UAV scheduling problem. The main novelty and contributions of this paper are summarized as follows.

- 1) Considering the energy consumption of UAVs and the characteristics of multi-stage tasks, a jointly trajectory planning and multi-stage task offloading problem model

is formulated for scheduling multiple UAVs with multiple pricing structures.

- 2) We design an effective heuristics FSGM for quickly constructing feasible solutions for the trajectory planning and task offloading of every UAV. The heuristics includes several innovative components designed specially for the problem under study. The swapping task insertion method guarantees the energy constraint during the task offloading process. The task insertion method inserts tasks into appropriate positions depending on the pheromone intensity. The boundary update method helps the task insertion method to avoid unnecessary trials. Besides, a probabilistic loop-breaking step is designed to balance the workload of UAVs.
- 3) Solution division method (SDM) is developed to further reconstructed to new solution by the solution division method. Solution division method (SDM) can help improve the diversity of solutions.
- 4) A bi-objective ant colony optimization (bi-ACO) framework is proposed to generate near-Pareto-optimal solution set. bi-ACO is an enhanced variant of general ACO. It is integrated with many effective components designed especially for the problem under study. The novelty of bi-ACO lies in the following aspects. (1) Multiple heterogeneous colonies are introduced with different preferences of objectives. (2) One pair of pheromone matrices is used for computing the probability of task selection. (3) Four pairs of pheromone matrices are used to determine the insertion position of a task. (4) The pheromone update method is employed with the delicately designed pheromone evaporation operation and the pheromone enhancement operation which fully consider different stage of tasks.

The rest of the paper is organized as follows. Section II gives the review of the related works. The problem under study is formulated in Section III. Section IV introduces the proposed algorithm and its major components. Delicate experiments are designed to compare the proposal and the baseline algorithms in Section V, followed by conclusions and future research in Section VI.

II. RELATED WORK

There have been many studies on UAV-assisted systems for these years. Deploying UAVs in these systems as the flying base stations [1], computation components [8], tracking agents [4] or logistics containers [9] provides a flexible and effective approach for many on-demand scenarios. The main research issues in these UAV-assisted systems can be classified into four categories: (1) computation task offloading problem, (2) UAV trajectory planning problem, (3) the hybrid problem of task offloading and UAV trajectory planning and (4) UAV placement/deployment problem.

In the UAV-assisted systems involving only the task offloading problem, the employed UAVs are usually quasi-static. In the case of quasi-static UAVs, the UAVs hover at fixed locations to provide computation or communication capabilities. They barely adjust the hovering locations, or periodically adjust the

hovering locations. Luan et al. [10] consider a joint optimization of topology reconstruction and subtask scheduling in the cellular Internet of UAV-assisted MEC emergency system. A hybrid subtask scheduling scheme is presented to make optimal task scheduling decisions. There is a few studies on how to determine the appropriate hovering locations depending on the task allocation decision. Du et al. [11] introduce the UAV-enabled wirelessly-powered MEC system for IoT devices. They relax the task offloading and path planning problems into a convex problem and apply the flow-shop scheduling techniques to address it. Zhang et al. [12] explore the task offloading problem in an UAV-assisted edge computing system, where a number of ground mobile devices are served by a flying UAV and a ground base station. A game theory-based scheme is constructed and the existence of Nash equilibrium is proved. Liao et al. [13] establish a new UAV swarm-enabled MEC network where UAVs are deployed to support time-constrained and low-resource unmanned surface vehicle communication and computation. The offloading decision is formulated as mixed-integer non-linear programming and solved by a modified deferred acceptance algorithm.

UAV trajectory planning problem usually exists in the surveillance, damage assessment, delivery or search and rescue operations where there are a given set of target points to visit by a squad of UAVs. Wang et al. [14] introduce a low-altitude UAV platform as both a mobile data collector and an aerial anchor node to assist in data collection and device positioning. They develop an efficient differential evolution based method to obtain the UAV trajectory. Bartolini et al. [15] investigate the trajectory planning problem in the monitoring critical scenarios requiring early anomaly discovery and intervention. They propose an efficient polynomial algorithm, called Greedy and Prune, with the objective of maximizing weighted progressive coverage. Sun et al. [16] use an efficient ant colony-based algorithm to optimize the flight path of UAVs in the scenario of vessel monitoring. Rigas et al. [17] schedule a set of UAVs across a graph where the nodes need to be visited multiple times at pre-defined points in time. A greedy heuristics and an ant colony optimization algorithm are designed for the problem. Huang et al. [9] study the dynamic task scheduling and path planning problem in the UAV-based intelligent on-demand meal delivery system. The Roulette-based flight dispatching approach is integrated into the simulated annealing based local search method to optimize the solutions. Hu et al. [18] investigate RIS-assisted UAV-borne IoT platform under jamming. Without utilizing the channel state information (CSI), they apply a deep reinforcement learning based method to learn the trajectory of UAV as well as its reconfigurable intelligent surface (RIS) configuration to better eliminate the jamming. Kang et al. [19] design a UAV-assisted data fusion method to efficiently collect data from IoT devices. First, the method gathers the data to clustering heads (CHs) nodes in the ground. Then the paths of multiple UAVs are optimized to collect data from CHs by formulating the problem to a multi-UAV problem.

There are also some studies on the UAV trajectory planning in the UAV-assisted networks, where the UAV trajectory planning problem is critical to guarantee communication. Zhao et al. [20] investigate the UAV-assisted network where the UAV and

base station serve ground users simultaneously. The method of block coordinate descent is applied for the UAV trajectory planning. Pang et al. [21] introduce the UAV into the wireless networks in order to enhance the secure transmission. An iterative heuristics is employed to solve the problem. Prasad and Ramkumar [22] consider the trajectory planning problem in a relay-based UAV-assisted cooperative communication for emergency applications. UAVs are used as both static base stations (cluster UAVs) and mobile base stations (relay UAVs). The Dijkstra algorithm is used to determine the optimal trajectory of UAVs. Eldeeb et al. [23] formulate an optimization problem in an IoT network, while minimizing the Age of Information (AoI) and energy consumption. They apply a deep reinforcement learning algorithm for the considered problem.

Many complex UAV-assisted systems need to solve both the task offloading and UAV trajectory planning problems. Most of the problems involve only one UAV and a set of tasks distributed in a given region. Sun et al. [24] jointly optimize the trajectory and CPU frequency of a fixed-wing UAV, and the offloading schedule to minimize the energy consumption of the UAV. An alternating optimization- and a successive convex approximation-based algorithms are developed to achieve the globally optimal linear trajectory, CPU configuration, and offloading schedule. Lin et al. [25] address the issue of how to effectively serve a 3-D wireless rechargeable sensor network with one UAV. They reduce the problem into a sub-modular maximization problem with routing constraints and present a cost-efficient algorithm. There are a few of studies on the trajectory planning and task scheduling for multiple UAVs. Miao et al. [26] raise a multi-UAVs-assisted MEC offloading algorithm for matching the dynamic mobile devices and UAV trajectory. In their proposal, first, a drone swarm scheduling and allocation strategy is developed to minimize the global flight length and energy consumption. Second, the optimal communication coverage of an UAV is computed to maximize the number of offloading services and minimize the total latency in completing the computation task. Finally, an UAV cluster computation offloading strategy with optimized energy efficiency is implemented. Khochare et al. [27] investigate a mission scheduling problem which co-schedules the UAV flight routes to visit and record video at waypoints, and their subsequent on-board edge analytics. They formulate the considered problem into a mixed integer linear programming problem and solve it accordingly. Tian et al. [28] study the service satisfaction-oriented task offloading and UAV scheduling problem for UAV-enabled MEC networks. Two genetic algorithms are proposed to solve the UAV scheduling sub-problem and the task offloading sub-problem, respectively.

The UAV placement or deployment problem exists in scenarios where the mobile network infrastructure is unable to provide wireless coverage. Yu et al. [29] study the placement problem of the UAV-mounted base stations. A backhaul-aware bandwidth allocation and an UAV placement algorithm are designed to efficiently solve the problem. Luo et al. [30] introduce the cached-enabled UAVs to provide on-demand content services for ground users. A weighted K-means method is employed for the UAV deployment problem. Liu et al. [31] study the UAV placement problem with the objective of maximizing

the fair coverage versus energy consumption while satisfying the backhaul constraints. An accurate and efficient proximal stochastic gradient descent based alternating algorithm is proposed to optimize the UAV locations. Fazel et al. [32] use the fast global K-means algorithm to determine the number of UAVs and their horizontal placements. Then, the altitudes of UAVs are optimized by the interior-point method.

In our previous study [33], we investigate the task scheduling and trajectory planning problem in the UAV-enabled Mobile Edge-Computing system with the objectives of minimizing the cost and the completion time. A NSGA-II algorithm is proposed for the problem. In this paper, we investigate a similar problem and take our previous work as one of the baseline studies. Our simulation results will show that the proposed method in the paper outperforms the benchmark algorithms either in effectiveness or efficiency.

III. PROBLEM DESCRIPTION

The notations used for the problem formulation are given in Table I.

In the considered problem, there are n SDs distributed in the given area. Each SD collects nearby data and generates one computation requirement periodically. Each computation requirement is taken as a task to be processed. The set $\mathbb{T} = \{T_1, \dots, T_n\}$ includes the tasks to process in one period. We assume the release times of these tasks are the same and given. The computation resources considered are homogeneous, which means the computation and communication times of tasks are resource-independent. Therefore, task T_i can be described by a 5-tuple $\langle DT(T_i), P(T_i), RT(T_i), L(T_i), D(T_i) \rangle$, where $DT(T_i)$ is the data transmission time, $P(T_i)$ is the task processing time, $RT(T_i)$ is the result transmission time and $L(T_i)$ is the location of T_i . In order to guarantee the data freshness, task T_i is assigned a deadline $D(T_i)$. The computation resources are UAVs equipped with GPS, WiFi and computing modules. There are two types of UAVs: reserved UAVs and on-demand UAVs. $\mathbb{U} = \{U_1, \dots, U_m\}$ contains m reserved UAVs. Reserved UAVs can be taken as the computation resources with long-term contracts with lower price α per time unit ε . $\mathbb{U}'$ contains employed on-demand UAVs. On-demand UAVs are temporary resources with higher unit price β per time unit ε . The average flying speed of UAVs is ν . The flying time from one location to another can be simply computed by $\frac{\mathbb{D}(L, L')}{\nu}$, where $\mathbb{D}(L, L')$ is the distance between two locations L and L' . Both reserved and on-demand UAVs are originally fully charged and provisioned in the base stations. We use $\mathbb{B}$ to denote the set of base stations. The initial location of U_j is $L^0(U_j) \in \mathbb{B}$. UAVs could swap batteries at any base station and the swapping operation is assumed to take only one time unit ε .

The energy consumption $\mathbb{P}(v)$ in watt (W) is defined as the function of the flying speed v (1). The energy consumption of rotary-wing UAVs consists of three components: blade profile, induced, and parasite power. For convenience, we employ all the constants and parameter settings directly from [34]. $\mathbb{P}(0)$ is the energy consumption by a hovering UAV. The maximum energy capacity of an UAV is $E_{\max}$. Note that the GPS, WiFi

TABLE I
NOTATIONS

Notation	Definition
n	Number of SDs/tasks
$\mathbb{T} = \{T_1, \dots, T_n\}$	Set of tasks
T_i	i^{th} task
$DT(T_i)$	Data transmission time of T_i
$P(T_i)$	Processing time of T_i
$RT(T_i)$	Result transmission time of T_i
$L(T_i)$	Location of T_i
$D(T_i)$	Deadline of T_i
m	Number of reserved UAVs
$\mathbb{U} = \{U_1, \dots, U_m\}$	Set of reserved UAVs
U_j	j^{th} UAV in $\mathbb{U}$
$\mathbb{U}'$	Set of on-demand UAVs
ε	Charging time unit of UAVs
α, β	Charging prices of reserved and on-demand UAVs per time unit ε
ν	Average flying speed of UAVs
$\mathbb{D}(L, L')$	Distance between locations L and L'
$\mathbb{B}$	Set of base stations
$L^0(U_j) \in \mathbb{B}$	Initial location of U_j
$\mathbb{P}(v)$	Energy consumption in watt with the flying speed v
$E_{\max}$	Maximum energy capacity of an UAV
$T_{i,1}, T_{i,2}, T_{i,3}$	Three sub-tasks of T_i
$S(T_{i,h})$	Starting time of $T_{i,h}$, $h = 1, 2, 3$
$C(T_{i,h})$	Completion time of $T_{i,h}$, $h = 1, 2, 3$
a_i	Task assignment of T_i
$U(T_i)$	UAV allocated to T_i
K_j	Number of sub-tasks assigned to U_j
$\pi_j = (\pi_{j,1}, \dots, \pi_{j,K_j})$	Task assignment of U_j
$\pi_{j,k}$	k^{th} task in π
T_{swap}	Swapping-battery task
$L_s(\pi_{j,k})$	Location of UAV at time $S(\pi_{j,k})$
$L_c(\pi_{j,k})$	Location of UAV at time $C(\pi_{j,k})$
$\mathbb{S} = \langle A, \Pi \rangle$	Schedule solution
A	Set of task assignments
Π	Set of UAV assignments
$E_j(t)$	Residual energy of U_j at time t
$TC(\mathbb{S})$	Total cost of $\mathbb{S}$
$CT(\mathbb{S})$	Completion time of $\mathbb{S}$

and computation modules on an UAV are equipped with their own power system. The energy consumption for computing and data transmission are not considered in the problem model.

$$\mathbb{P}(v) = \underbrace{0.044v^2 + 580}_{\text{blade profile}} + \underbrace{0.0073v^3}_{\text{parasite}} + \underbrace{790.67 \left(\sqrt{1 + \frac{v^4}{10750}} - \frac{v^2}{103.68} \right)^{\frac{1}{2}}}_{\text{induced}} \quad (1)$$

There are three stages for processing task T_i by an assigned UAV U_j as follows.

- Data Transmission (DT) stage. In this stage, U_j hovers on top of the corresponding SD and receives data of T_i .
- Data Computation (DC) stage. In this stage, U_j could hover near the SD or fly to another location.

- Result Transmission (RT) stage. In this stage, U_j hovers on top of the corresponding SD and sends the result of T_i to SD.

Sub-tasks $T_{i,1}$, $T_{i,2}$ and $T_{i,3}$ are introduced to represent three stages of T_i . Once a sub-task starts, it cannot be interrupted until it is completed. An UAV can only take one sub-task at a time and multi-thread mode is not considered. We employ $S(T_{i,h})/C(T_{i,h})$ to denote the starting/completion times of $T_{i,h}$ ($h = 1, 2, 3$).

An assignment a_i of T_i is represented by a 7-tuple $\langle U(T_i), S(T_{i,1}), C(T_{i,1}), S(T_{i,2}), C(T_{i,2}), S(T_{i,3}), C(T_{i,3}) \rangle$. $U(T_i)$ is the UAV that T_i is assigned to. Accordingly, the assignment π_j of U_j can be represented by a sub-task sequence $\pi_j = (\pi_{j,1}, \pi_{j,2}, \dots, \pi_{j,K_j})$, where $\pi_{j,k}$ ($k = 1, \dots, K_j$) could be a sub-task or a swapping-battery task T_{swap} . K_j is the number of sub-tasks (including swapping-battery tasks) assigned to U_j . We define $L_s(\pi_{j,k})$ and $L_c(\pi_{j,k})$ as the locations at the times of $S(\pi_{j,k})$ and $C(\pi_{j,k})$, respectively. $S(\pi_{j,k})/C(\pi_{j,k})$ is the starting/completion time of $\pi_{j,k}$.

The schedule solution $\mathbb{S} = \langle A, \Pi \rangle$ of the considered problem is composed of the set of task assignments $A = \{a_i | i = 1, \dots, n\}$ and the set of UAV assignments $\Pi = \{\pi_j | U_j \in \mathbb{U} \cup \mathbb{U}'\}$. The schedule is feasible if and only if the following constraints are satisfied.

- Precedence constraint limits that a task should be processed in order of stages. Equ. (2) restricts that a stage can start only when its previous stage is completed.
- Location constraint is defined to restrict the motion of UAVs. When an UAV is processing the 1st or 3rd stage sub-task, it must hover at the location of the sub-task until the sub-task is completed. The constraint is defined by (3).
- Priority constraint restricts that the sub-tasks assigned to the same UAV should be processed in order of the sub-task sequence. A sub-task $\pi_{j,k}$ in π_j can start only when $\pi_{j,k-1}$ is completed. The constraint is defined by (4)–(5). Equ. (5) implies that the first sub-task of an UAV cannot start until the assigned UAV arrives at the location of the sub-task.
- Deadline constraint means that the completion time of a task cannot exceed its deadline. It is defined by (6).
- Non-overlapping constraint makes sure that the processing times of sub-tasks on the same UAV cannot overlap in time. The constraint can be defined by (7).
- Energy constraint guarantees each UAV has sufficient energy at any time during task execution. We use $E_j(t)$ to represent the residual energy of U_j at time t . $E_j(t)$ cannot be negative at any time. The energy constraint is defined by (8)–(15). Equ (8) indicates the residual energy of U_j at the starting time of $\pi_{j,k}$ cannot be negative. The residual energy of U_j at the completion time of each sub-task cannot be negative as well (9). In order to compute $E_j(S(\pi_{j,k}))$ and $E_j(C(\pi_{j,k}))$, we introduce two new notations: $\Delta E_{j,k}$ to represent the energy consumption in the duration of sub-task $\pi_{j,k}$ and $\Delta G_{j,k}$ to represent the energy consumption during the time gap between two consecutive sub-tasks. There would be time gap between two consecutive sub-tasks. The time gap takes place when the UAV is idle and on the way flying to the next sub-task.

Equ (10) means the energy difference of U_j between the starting and completion time of $\pi_{j,k}$ is $\Delta E_{j,k}$. Equ (11) means the energy consumed during the time gap between $C(\pi_{j,k-1})$ and $S(\pi_{j,k})$ is $\Delta G_{j,k}$. $\Delta E_{j,k}$ depends on the type of $\pi_{j,k}$ (12). If $\pi_{j,k}$ is T_{swap} , U_j would be fully charged at time $C(\pi_{j,k})$. If $\pi_{j,k}$ is the 1st or 3rd stage sub-task, the energy would be consumed based on the hovering duration. Otherwise, the consumed energy would include two parts: flying and hovering energy consumption. The flying time $t_{j,k}^{fly}$ of U_j during processing $\pi_{j,k}$ depends on the distance it moves (13). Since the total duration of $\pi_{j,k}$ is $P(\pi_{j,k})$, the hovering time $t_{j,k}^{hover}$ during processing $\pi_{j,k}$ can be computed by (14). $\Delta G_{j,k}$ can be computed by (15).

$$C(T_{i,1}) \leq S(T_{i,2}), C(T_{i,2}) \leq S(T_{i,3}) \quad (2)$$

$$i = 1, \dots, n$$

$$L_s(T_{i,h}) = L_c(T_{i,h}) = L(T_{i,h}), h = 1, 3 \quad (3)$$

$$S(\pi_{j,k}) \geq C(\pi_{j,k-1}) + \frac{\mathbb{D}(L_c(\pi_{j,k-1}), L_s(\pi_{j,k}))}{\nu} \quad (4)$$

$$S(\pi_{j,1}) \geq \frac{\mathbb{D}(L^0(U_j), L(\pi_{j,1}))}{\nu} \quad (5)$$

$$C(T_i) \leq D(T_i) \quad (6)$$

$$\min\{C(T_{i,h}), C(T_{i',h})\} \leq \max\{S(T_{i,h}), S(T_{i',h})\} \quad (7)$$

$$U(T_i) = U(T_{i'}), h = 1, 2, 3$$

$$E_j(C(\pi_{j,k})) \geq 0 \quad (8)$$

$$E_j(S(\pi_{j,k})) \geq 0 \quad (9)$$

$$E_j(C(\pi_{j,k})) = E_j(S(\pi_{j,k})) - \Delta E_{j,k} \quad (10)$$

$$E_j(S(\pi_{j,k})) = E_j(C(\pi_{j,k-1})) - \Delta G_{j,k} \quad (11)$$

$$\Delta E_{j,k} = \begin{cases} E_j(S(\pi_{j,k})) - E_{\max}, & \pi_{j,k} = T_{swap} \\ \mathbb{P}(0)DT(T_i), & \pi_{j,k} = T_{i,1} \\ \mathbb{P}(0)RT(T_i), & \pi_{j,k} = T_{i,3} \\ \mathbb{P}(\nu)t_{j,k}^{fly} + \mathbb{P}(0)t_{j,k}^{hover}, & \pi_{j,k} = T_{i,2} \end{cases} \quad (12)$$

$$t_{j,k}^{fly} = \frac{\mathbb{D}(L_s(\pi_{j,k}), L_c(\pi_{j,k}))}{\nu} \quad (13)$$

$$t_{j,k}^{hover} = P(\pi_{j,k}) - t_{j,k}^{fly} \quad (14)$$

$$\Delta G_{j,k} = \mathbb{P}(\nu) \times \frac{\mathbb{D}(L_c(\pi_{j,k-1}), L_s(\pi_{j,k}))}{\nu} \quad (15)$$

Fig. 2 gives examples of two solutions. There are three tasks to schedule and two UAVs provisioned in two base stations respectively in Fig. 2. The information of tasks is shown in Table II. In the solution $\mathbb{S}_1$ (Fig. 2(a)), only U_1 is employed. Its assignment π_1 is $(T_{1,1}, T_{1,2}, T_{2,1}, T_{2,2}, T_{3,1}, T_{3,2}, T_{swap}, T_{1,3}, T_{2,3}, T_{3,3}, T_{swap})$. The Gantt chart of the solution is shown in Fig. 3(a). Table III gives the detail of the task assignments. In the solution $\mathbb{S}_2$ (Fig. 2(b)), two UAVs U_1 and U_2 are employed. The assignment π_1 is $(T_{1,1}, T_{1,2}, T_{2,1}, T_{2,2}, T_{2,3}, T_{1,3}, T_{swap})$. The assignment π_2 is $(T_{3,1}, T_{3,2}, T_{3,3}, T_{swap})$. The Gantt chart of the solution is

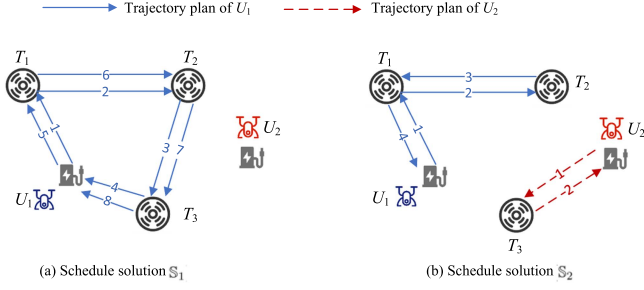


Fig. 2. Examples of solutions.

TABLE II
INFORMATION OF TASKS

	$DT(T_i)$	$P(T_i)$	$RT(T_i)$	$D(T_i)$
T_1	2	5	1	100
T_2	3	3	1	100
T_3	1	7	1	100

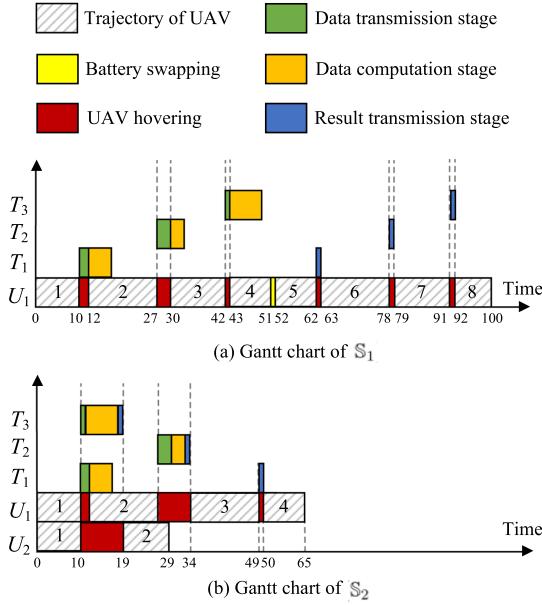


Fig. 3. Gantt charts.

TABLE III
TASK ASSIGNMENTS OF SOLUTION 1

	$U(T_i)$	$S(T_{i,1})$	$C(T_{i,1})$	$S(T_{i,2})$	$C(T_{i,2})$	$S(T_{i,3})$	$C(T_{i,3})$
T_1	U_1	10	12	12	17	62	63
T_2	U_1	27	30	30	33	78	79
T_3	U_1	42	43	43	50	91	92

shown in Fig. 3(b). Table IV gives the details of the task assignments.

Given a feasible S , its total cost $TC(S)$ and the completion time $CT(S)$ can be computed by (16)–(17).

$$TC(S) = \alpha \sum_{j \in \mathbb{U}} C(\pi_{j,K_j}) + \beta \sum_{j \in \mathbb{U}'} C(\pi_{j,K_j}) \quad (16)$$

TABLE IV
TASK ASSIGNMENTS OF SOLUTION 2

	$U(T_i)$	$S(T_{i,1})$	$C(T_{i,1})$	$S(T_{i,2})$	$C(T_{i,2})$	$S(T_{i,3})$	$C(T_{i,3})$
T_1	U_1	10	12	12	17	49	50
T_2	U_1	27	30	30	33	33	34
T_3	U_2	10	11	11	18	18	19

$$CT(S) = \max_{i=1}^n C(T_i) \quad (17)$$

As the solutions S_1 and S_2 shown in Fig. 3, S_1 employs only one reserved UAV U_1 , but S_2 employs an extra on-demand UAV U_2 . Suppose $\alpha = 1$ and $\beta = 4$, we have $TC(S_1) = 92$ and $TC(S_2) = 120$. Apparently, S_1 tasks much less cost than that of S_2 . However, the reserved UAVs may be overloaded with less on-demand UAVs and the completion time of all tasks may be greatly increased. Taking the solutions in Fig. 3 as an example, $CS(S_1) = 92$ and $CS(S_2) = 44$. Although S_2 has more cost, its completion time is greatly reduced. We introduce Pareto optimization for the considered problem. The paper aims to obtain the Pareto front of the considered problem.

IV. PROPOSED METHOD

In the paper, we employ the bi-criterion ant colony optimization (bi-ACO) algorithm for the bi-objective optimization problem under study. We first introduce the main framework of bi-ACO, followed by the details for every key operations and settings.

A. Main Framework of bi-ACO

Algorithm 1 describes the major framework of the proposal.

$\mathcal{N}$ ant colonies $CO_1, CO_2, \dots, CO_N$ are initially constructed with initial pheromone matrices (Lines 1-2). A global Pareto set PF^* is maintained to accommodate non-dominated solutions and it is initially empty (Line 3). Then an iterative process (Lines 4-13) is repeated until the termination condition is met, i.e., $I_{\max}$ generations have been done. In each generation, three steps (Lines 7-12) are performed by every ant in each colony:

- 1) Each ant constructs a feasible solution based on pheromone matrices and heuristic values (Line 7).
- 2) PF^* is updated accordingly (Line 8).
- 3) If the obtained solution is a non-dominated one, it is modified to generate a promising solution (Lines 9-12). The feasibility of the promising solution is checked and PF^* is updated accordingly.

At the end of each generation, the pheromone matrices of all colonies are updated according to PF^* (Line 13).

As can be seen, the proposed bi-ACO algorithm consists of several major settings and components: the settings of heterogeneous colonies to define the pheromone matrices and heuristic matrices, the Feasible Solution Generation Method (FSGM) to generate a feasible solution, the Solution Division Method (SDM) to generate a promising solution and the Pheromone

Algorithm 1: bi-ACO Framework.

```

1 Initialize  $\mathcal{N}$  ant colonies  $CO_1, CO_2, \dots, CO_N$ ;
2 Initialize pheromone matrices for  $\mathcal{N}$  colonies;
3  $PF^* \leftarrow \emptyset$ ; /*  $PF^*$  is the Pareto Front */
4 while Termination condition is not met do
5   for  $p = 1, 2, \dots, \mathcal{N}$  do
6     for  $q = 1, 2, \dots, n_a$  do
7       Call Feasible Solution Generation Method to generate a
       feasible solution  $S_1$  based on pheromone matrices
       and heuristic values ;
8       Update  $PF^*$  by  $S_1$ ;
9       if  $S_1 \in PF^*$  then
10        Perform Solution Division Method on  $S_1$  to
        generate a promising solution  $S_2$ ;
11        if  $S_2$  is feasible then
12          Update  $PF^*$  by  $S_2$ ;
13   Call Pheromone Update Method to update pheromone
   matrices of all colonies according to  $PF^*$ ;
14 return  $PF^*$ .
```

Update Method (PUM) to update pheromone matrices. The details of these settings and components are given in the following sub-sections.

B. Colony Settings

There are $\mathcal{N}$ ant colonies $CO_1, CO_2, \dots, CO_N$. CO_p is the p^{th} colony in the ant colony system. p is the index of a colony. Each colony has n_a ants $an_1, an_2, \dots, an_{n_a}$. an_q is the q^{th} ant in a colony. q is defined as the index of an ant. In each generation, an ant an_q of CO_p makes the task offloading decisions based on the pheromone and heuristic values. One task offloading decision includes two parts: a task to select and its insertion position in the sequence. Each colony maintains five pairs of pheromone matrices where one pair is used for computing the probability of selecting a task and the other four pairs are used to determine the insertion position of a task.

The probability of an unscheduled task to be selected is defined in (18) which is shown at the bottom of this page. In the equation, T_i is the previously selected task and $T_{i'}$ denotes a candidate task about to be selected. Suppose W is the set of tasks which have been assigned to the current UAV and $\bar{W}$ is the set of unscheduled tasks. $\eta_{i,i'}$ (19) is the heuristic information corresponding to the total cost objective and $\hat{\eta}_{i,i'}$ (20) is the heuristic information corresponding to the completion time objective. In (20), $ST(T_{i'})$ is defined as the slot time of $T_{i'}$. It can be computed by (21). $(\tau_{i,i'})$ is the selection pheromone matrix corresponding to the total cost objective and $(\hat{\tau}_{i,i'})$ is the selection pheromone matrix corresponding to the completion time objective. All items in the selection pheromone matrices are initialized to 1. In order to promote the ants to search with different preferences of objectives, every ant in the colony has different relative importance factors for the objectives. The relative importance factor $\lambda_{p,q}$ for the ant an_q in the colony CO_p

is defined by (22).

$$\eta_{i,i'} = \left(\frac{\max\{\mathbb{D}(L(T_{i'}), L(T_{i''})) | T_{i''} \in W\}}{\nu} \right)^{-1} \quad (19)$$

$$\hat{\eta}_{i,i'} = \left(ST(T_{i'}) - \frac{\mathbb{D}(L(T_i), L(T_{i'}))}{\nu} \right)^{-1} \quad (20)$$

$$ST(T_{i'}) = D(T_{i'}) - DT(T_{i'}) - P(T_{i'}) - RT(T_{i'}) \quad (21)$$

$$\lambda_{p,q} = \frac{2(q-1)}{N^2-1} + \frac{p-1}{N+1} \quad (22)$$

In particular, the probability of the first task to select by an UAV is defined by (23) which is shown at the bottom of the next page, where T_0 is a dummy task.

Every colony maintains four pairs of insertion pheromone matrices $(\chi_{i,i'}^f)$ and $(\hat{\chi}_{i,i'}^f)$ ($f = 1, 2, 3, 4$), where $(\chi_{i,i'}^f)$ is the insertion pheromone matrix corresponding to the total cost objective and $(\hat{\chi}_{i,i'}^f)$ is the insertion pheromone matrix corresponding to the completion time objective. Similar to the selection pheromone matrices, all items in these insertion pheromone matrices are initialized to 1 and updated generation by generation. The reason that we need 4 pairs of matrices for task insertion is that there are 4 possible scenarios for an insertion position in the task insertion method. Given an insertion position, the closest transmission sub-tasks on its left and right sides may be 1) both the first-stage subtasks, 2) both the third-stage subtasks, 3) the first-stage sub-task and third-stage sub-task, or 4) the third-stage sub-task and first-stage sub-task. The usage of these matrices will be introduced in the following sub-sections.

C. Feasible Solution Generation Method

The q^{th} ant of CO_p constructs a feasible solution by FSGM (Algorithm 2). There are three major steps in FSGM: First, the ant generates the task and UAV assignments for all reserved UAVs (Lines 3-27). Second, if all reserved UAVs are used but there are still unscheduled tasks, the on-demand UAVs are employed to undertake the rest tasks (Lines 28-32). Third, the ant improves the obtained solution by swapping assignments of some UAVs (Lines 33-36). The following subsections will introduce the major steps of FSGM and the details of key components.

1) *Major Steps of FSGM*: In each iteration (Lines 4-27) of the first step, the ant constantly tries to assign tasks to the current UAV until the termination conditions are met. There are three termination conditions: (1) all tasks have been scheduled (Line 8), (2) the boundary parameter becomes invalid (Line 23) and (3) the ant stops assigning at a given probability (Line 25). The boundary parameter B is used to restrict the task insertion positions in the Task Insertion Method.

The ant uses $\bar{W}_j$ to preserve all candidate tasks to try. $\bar{W}_j$ contains all unscheduled tasks at the beginning of each trial (Line 9). Then the ant iteratively performs the following process

$$P(p, q, T_i, T_{i'}) = \frac{((\tau_{i,i'})^{\lambda_{p,q}} (\hat{\tau}_{i,i'})^{(1-\lambda_{p,q})})^\phi \cdot ((\eta_{i,i'})^{\lambda_{p,q}} (\hat{\eta}_{i,i'})^{(1-\lambda_{p,q})})^\varphi}{\sum_{T_{i''} \in \bar{W}} ((\tau_{i,i''})^{\lambda_{p,q}} (\hat{\tau}_{i,i''})^{(1-\lambda_{p,q})})^\phi \cdot ((\eta_{i,i''})^{\lambda_{p,q}} (\hat{\eta}_{i,i''})^{(1-\lambda_{p,q})})^\varphi} \quad (18)$$

Algorithm 2: Feasible Solution Generation Method
FSGM(p, q).

```

1  $\bar{W} \leftarrow \mathbb{T};$   $\bar{W}$  contains all tasks initially  $*/$ 
2  $A \leftarrow \emptyset; \Pi \leftarrow \emptyset; \mathbb{U}' \leftarrow \emptyset;$ 
3 for  $\forall U_j \in \mathbb{U}$  do
4    $A_j \leftarrow \emptyset;$   $A_j$  contains assignments of tasks
     selected by  $U_j$   $*/$ 
5    $\pi_j \leftarrow ();$  Sub-task sequence is initially
     empty  $*/$ 
6    $T \leftarrow T_0;$   $T$  is the previous selected task  $*/$ 
7    $B \leftarrow 1;$   $B$  is the boundary parameter
     restricting the task insertion operation  $*/$ 
8   while  $\bar{W}$  is not empty do
9      $\bar{W}_j \leftarrow \bar{W};$   $\bar{W}_j$  preserves candidate tasks  $*/$ 
10    for  $R = 1$  to  $R_{max}$  do
11      Compute  $P(p, q, T, T')$  for  $\forall T' \in \bar{W}_j;$ 
12       $T_i \leftarrow RWS(P(p, q, T, T')|_{T' \in \bar{W}_j});$   $RWS$  is
        Roulette Wheel Selection  $*/$ 
13       $\langle \pi'_j, A'_j \rangle \leftarrow TIM(A_j, \pi_j, T_i, B, p, q);$   $TIM$  is
        Task Insertion Method  $*/$ 
14      if  $\langle \pi'_j, A'_j \rangle$  is feasible then
15         $\pi_j \leftarrow \pi'_j;$ 
16         $A_j \leftarrow A'_j;$ 
17         $\bar{W} \leftarrow \bar{W} - \{T_i\};$ 
18         $T \leftarrow T_i;$ 
19        break;
20      else
21         $\bar{W}_j \leftarrow \bar{W}_j - \{T_i\};$ 
22       $B \leftarrow BUM(\pi_j, B);$   $BUM$  is Boundary Update
        Method  $*/$ 
23      if  $B = -1$  then
24        break;
25      break at the probability of  $\frac{\theta_b \times (p-1)}{N-1};$ 
26       $A \leftarrow A + A_j;$ 
27       $\Pi \leftarrow \Pi + \{\pi_j\};$ 
28  $*/$  On-demand UAVs are employed for the
     unscheduled tasks  $*/$ 
29 while  $\bar{W}$  is not empty do
30    $\mathbb{U}' \leftarrow \mathbb{U}' + \{U\};$   $*/$  Employ an on-demand UAV  $U$   $*/$ 
31   Generate task assignment set  $A'$  and UAV assignment  $\pi'$  in
     the similar manner as steps 8-25;
32    $A \leftarrow A + A';$ 
33    $\Pi \leftarrow \Pi + \{\pi'\};$ 
34 for  $\forall U_j \in \mathbb{U}$  and  $\forall U_{j'} \in \mathbb{U}'$  do
35   if  $L^0(U_j) = L^0(U_{j'})$  then
36     if  $C(\pi_j, K_j) < C(\pi_{j'}, K_{j'})$  then
37       Swap the task and UAV assignments of  $U_j$  and  $U_{j'};$ 
38 return  $\langle A, \Pi \rangle.$ 

```

(Lines 10-21) at most R_{max} times unless a feasible assignment is obtained.

- 1) The probability of every candidate task to be selected is computed (Line 11).
- 2) Roulette Wheel Selection is performed and one task is selected (Line 12).

- 3) Task Insertion Method tries to insert the selected task into the current sub-task sequence of U_j (Line 13).
- 4) If the selected task is successfully inserted into the sub-task sequence (Line 14), the current sub-task sequence and task assignment are updated accordingly (Lines 15-16). The task is removed from the set of unscheduled tasks (Line 17) and it is marked as the previous selected task (Line 18). The current trial is completed (Line 19).
- 5) If the task insertion fails, the selected task is removed from the candidate set (Line 21) and the next iteration will begin.

Whether the above process successfully assigns a task to U_j or not, the boundary parameter will be updated according to the current sub-task sequence (Line 22). The ant would take a new trial of assigning task with the updated B . However, if the updated B is invalid, i.e., $B = -1$, then the ant would stop assigning tasks to the current UAV. Besides, the ant stops assigning at a given probability which is computed based on the index of the colony (Line 25). Without the probability condition for termination, an UAV would be assigned as many tasks as possible. But with the condition, some reserved UAVs may be not fully arranged and leave more tasks to on-demand UAVs. In other words, the lower the probability of breaking the loop (Lines 8-25), the fuller the reserved UAVs and the lower the cost. The colonies with smaller indexes have more preference towards the objective of cost and the colonies with larger indexes prefer to the objective of completion time. Therefore, we compute the probability condition based on the index of the colony, i.e., p . The colonies with smaller indexes will gain smaller probability of breaking the loop, vice versa. Besides, we introduce a breaking factor θ_b to manipulate the probability of breaking the loop. $\theta_b = 0$ indicates no breaking. The higher θ_b is, the higher probability the algorithm will break the loop. θ_b will be calibrated in the experiment section.

The second step (Lines 28-32) takes place if the reserved UAVs are not sufficient to handle all tasks. On-demand UAVs are randomly selected and take over the rest of tasks. The assignments of on-demand UAVs are generated in the similar manner as those of the reserved ones.

In the third step, the ant improves the obtained solution by swapping assignments of some UAVs (Lines 33-36). For every pair of UAVs with the same initial location but different types, the completion times of them are compared. If an on-demand UAV works more time than that of a reserved one, swapping the assignments between them can lead to lower cost.

There are two key components in FSGM: (1) Task Insertion Method inserts task into appropriate position of sub-task sequence for a given UAV, and (2) Boundary Update Method shrinks insertion position boundary in order to improve efficiency. The details of the components are given as follows.

2) *Task Insertion Method*: Algorithm 3 details how to insert the selected task into the current sub-task sequence. Task Insertion Method (TIM) tries to insert three stages of a given

$$P(p, q, T_0, T_i) = \frac{((\tau_{0,i})^{\lambda_{p,q}} (\hat{\tau}_{0,i})^{(1-\lambda_{p,q})})^\phi \cdot ST(T_i)^{-\varphi}}{\sum_{T_{i'} \in \bar{W}} ((\tau_{0,i'})^{\lambda_{p,q}} (\hat{\tau}_{0,i'})^{(1-\lambda_{p,q})})^\phi \cdot ST(T_{i'})^{-\varphi}} \quad (23)$$

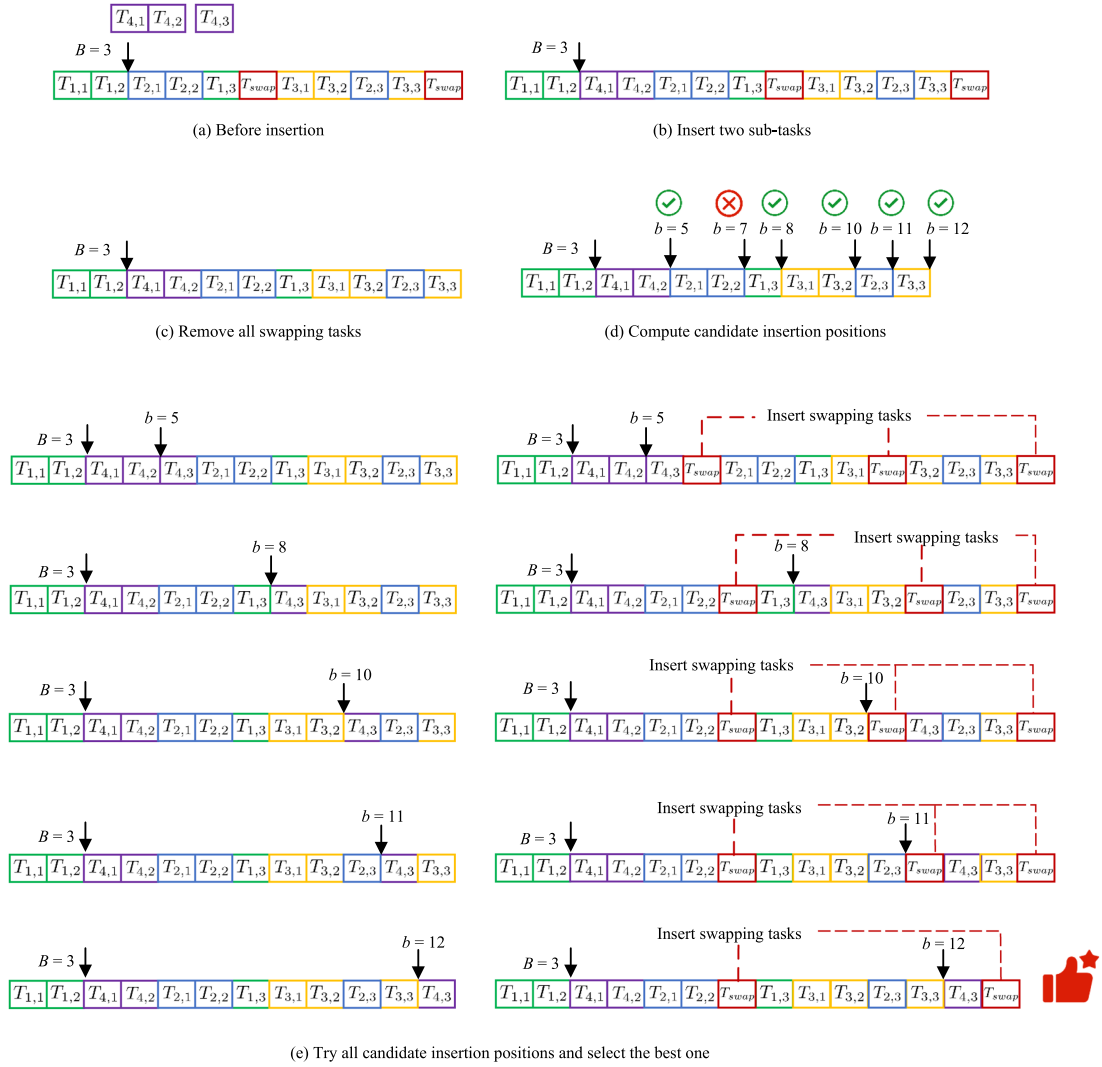


Fig. 4. Example of TIM procedure.

task T_i into the current sub-task sequence π_j of U_j with the given insertion boundary B . TIM may fail to obtain a feasible solution, therefore, it operates on the copy of input π_j (Line 1). If T_i is the first task assigned to U_j (Line 2), the sub-tasks of T_i are inserted sequentially (Line 3). T_{swap} is arranged by Swapping Task Insertion Method (Line 4). The corresponding task assignments are computed (Line 5). The obtained solution is returned if it is feasible (Lines 6-7). If T_i is not the first task to insert, then the following steps are performed.

- 1) Two sub-tasks $T_{i,1}$ and $T_{i,2}$ are inserted into the sub-task sequence from position B (Lines 8-9). We assume a task starts the computation stage immediately when the data transmission stage is completed.
- 2) Delete all T_{swap} sub-tasks from position $B + 2$ to the end (Line 10). $T_{i,3}$ may be inserted into any position. T_{swap} sub-tasks arranged previously may become inappropriate. Therefore, it is necessary to remove these T_{swap} sub-tasks and rearrange them afterwards. After the insertion and delete operations, π'_j is assumed to have K_j sub-tasks in total, i.e., $\pi'_j = (\pi'_{j,1}, \pi'_{j,2}, \dots, \pi'_{j,K_j})$

- 3) The pheromone intensity of the every possible position b (from $B + 2$ to K_j) is evaluated by PheromoneIntensity function (Lines 11-16). Set $\mathcal{B}$ is used to deposit all candidate positions and their pheromone intensities (Line 11). $T_{i,3}$ must be inserted after position $B + 1$. If the sub-task at position b is a 2^{nd} stage sub-task, $T_{i,3}$ cannot inserted before it, since we assume the first two stages of the same task must be executed continuously (Lines 13-14).
- 4) All candidate positions are sorted in descending order of pheromone intensities and only the top five with the highest intensities are kept (Lines 17-18).
- 5) $T_{i,3}$ are inserted into the position which can lead to the feasible solution and the completion time is minimal (Lines 19-32).
- 6) If no feasible solution can be obtained, the algorithm returns NULL (Lines 33-34).

Fig. 4 shows an example of TIM procedure. The current sub-task sequence is given as Fig. 4(a) and T_4 is about to insert into it. Sub-tasks $T_{4,1}$ and $T_{4,2}$ are inserted at position B and $B + 1$ as shown in Fig. 4(b). Then all swapping tasks are

Algorithm 3: Task Insertion Method TIM(A_j, π_j, T_i, B, p, q).

```

1  $\pi'_j \leftarrow \pi_j$ ; /*  $\pi'_j$  is a copy of  $\pi_j$  */
2 if  $A_j = \emptyset$  then
3    $\pi'_j \leftarrow (T_{i,1}, T_{i,2}, T_{i,3})$ ;
4   Check the feasibility of  $\pi'_j$  on the energy constraint and
   insert  $T_{swap}$  at appropriate positions by Swapping Task
   Insertion Method;
5   Compute the task assignments  $A'_j$  for  $\pi'_j$ ;
6   if  $A'_j$  and  $\pi'_j$  are feasible then
7     return  $\langle \pi'_j, A'_j \rangle$ ;
8 Insert  $T_{i,1}$  at the position  $B$  of  $\pi'_j$ ;
9 Insert  $T_{i,2}$  at the position  $B + 1$  of  $\pi'_j$ ;
10 Delete all  $T_{swap}$  in  $\pi'_j$  starting from the position  $B + 2$  to the
    end;
    /* Suppose  $\pi'_j = (\pi'_{j,1}, \pi'_{j,2}, \dots, \pi'_{j,K_j})$  */
11  $\mathcal{B} \leftarrow \emptyset$ ; /*  $\mathcal{B}$  deposits candidate insertion
    positions and their pheromone intensities */
12 for  $b = B + 2$  to  $K_j + 1$  do
13   if  $\pi'_{j,b}$  is a 2nd stage sub-task then
14     Continue;
15    $Pi_b \leftarrow PIM(\pi'_j, b, T_i, p, q)$ ; /* PIM is Pheromone
    Intensity Method */
16    $\mathcal{B} \leftarrow \mathcal{B} + \langle b, Pi_b \rangle$ ;
17 Sort all items in  $\mathcal{B}$  in descending order of pheromone intensities;
18 Remove all items from  $\mathcal{B}$  except the top five items with the
    highest intensities;
19  $A_j^* \leftarrow \emptyset$ ;
20  $\pi_j^* \leftarrow ()$ ;
21  $CT^* \leftarrow +\infty$ ;
22 for  $\langle b, Pi_b \rangle \in \mathcal{B}$  do
23    $\pi''_j \leftarrow \pi'_j$ ; /*  $\pi''_j$  is a copy of  $\pi'_j$  */
24   Insert  $T_{i,3}$  at the position  $b$  of  $\pi''_j$ ;
25   Check the feasibility of  $\pi''_j$  on the energy constraint and
   insert  $T_{swap}$  at appropriate positions by Swapping Task
   Insertion Method;
26   Compute the task assignments  $A''_j$  for  $\pi''_j$ ;
27   if  $A''_j$  and  $\pi''_j$  are feasible then
28     Compute the completion time  $CT$  of  $A''_j$ ;
29     if  $CT < CT^*$  then
30        $A_j^* \leftarrow A''_j$ ;
31        $\pi_j^* \leftarrow \pi''_j$ ;
32        $CT^* \leftarrow CT$ ;
33 if  $CT^* = +\infty$  then
34   return NULL;
35 return  $\langle \pi_j^*, A_j^* \rangle$ ;

```

removed (Fig. 4(c)). There are 6 candidate insertion positions ($b = 5, 7, 8, 10, 11, 12$) for inserting $T_{4,3}$ and their pheromone intensities are computed. Five candidate insertion positions ($b = 5, 8, 10, 11, 12$) with highest pheromone intensities are reserved (Fig. 4(d)). $T_{4,3}$ is tried to insert into every candidate position and every attempt is evaluated (Fig. 4(e)). Finally, $b = 12$ is the best insertion position for $T_{4,3}$.

There are two major components in Algorithm 3: (1) Swapping Task Insertion Method (STIM) to insert T_{swap} tasks to sub-task sequence and (2) Pheromone Intensity Method (PIM) to compute the pheromone intensity for a given insertion position. They are described as follows.

The key idea of STIM is to check if the residual energy of U_j at the end of a sub-task is sufficient enough to execute a

closest T_{swap} task. In the case of insufficient energy, a T_{swap} task should be inserted before the sub-task. The T_{swap} task to insert should be delicately selected. Since it will be inserted between two sub-tasks $\pi_{j,k-1}$ and $\pi_{j,k}$, the residual energy of U_j at the end of $\pi_{j,k-1}$ should be sufficient to fly to the location of T_{swap} .

In PIM, four steps are performed to compute the pheromone intensity for a given insertion position (Algorithm 4):

- 1) The recent transmission sub-task π_{j,b_1} before b is found first (Lines 6-14). π_{j,b_1} could either be a data transmission sub-task or a result transmission sub-task. h_1 and i_1 are employed to denote the stage index and task index of π_{j,b_1} .
- 2) Then the recent transmission sub-task π_{j,b_2} after b is found (Lines 15-23). h_2 and i_2 are employed to denote the stage index and task index of π_{j,b_2} .
- 3) It is possible that there is no transmission sub-task after b , then i_2 is $n + 1$ indicating a dummy task index. In this case, Pi is set to $+\infty$ (Lines 24-25);
- 4) Otherwise, Pi is computed based on different insertion pheromone matrices according to the values of h_1 and h_2 (Lines 26-33).

There are four pairs of pheromone matrices employed in Algorithm 4 to compute pheromone intensity (Lines 26-33). $(\chi_{i,i'}^1), (\chi_{i,i'}^2), (\chi_{i,i'}^3), (\chi_{i,i'}^4)$ are the insertion pheromone matrices corresponding to the total cost objective and $(\hat{\chi}_{i,i'}^1), (\hat{\chi}_{i,i'}^2), (\hat{\chi}_{i,i'}^3), (\hat{\chi}_{i,i'}^4)$ are the insertion pheromone matrices corresponding to the completion time. Items $\chi_{i,i'}^1$ and $\hat{\chi}_{i,i'}^1$ indicate the preference that $T_{i',3}$ inserts after $T_{i,1}$. Items $\chi_{i,i'}^2$ and $\hat{\chi}_{i,i'}^2$ indicate the preference that $T_{i',3}$ inserts before $T_{i,1}$. Items $\chi_{i,i'}^3$ and $\hat{\chi}_{i,i'}^3$ indicate the preference that $T_{i',3}$ inserts after $T_{i,3}$. Items $\chi_{i,i'}^4$ and $\hat{\chi}_{i,i'}^4$ indicate the preference that $T_{i',3}$ inserts before $T_{i,3}$.

3) *Boundary Update Method*: Algorithm 5 reveals how to update B . The more tasks have been assigned to the UAV in the previous iterations, the more possible insertion positions there would be. It would be low in efficiency to try all possible positions to insert a task. B is slowly increased with the number of tasks assigned to the current UAV. B is updated to the position of the recent transmission sub-task (Lines 1-3). If there is no transmission sub-task after position B , then an invalid position is returned (Lines 4-5).

As in Fig. 4, the final sub-sequence is the one where $T_{4,3}$ is inserted at position $b = 12$. Afterwards, B is updated to 5 by BUM.

D. Solution Division Method

Solution division method (SDM) is designed to reconstruct a given solution to a new one (as shown in Algorithm 6). The following major steps are performed by SDM:

- 1) The input schedule is copied (Line 1). The following operations are performed on the copy of the input schedule.
- 2) The UAV U_j with the maximum completion time is found (Line 2).
- 3) Tasks in U_j are picked up and put into set $\mathbb{T}'$ at a 50% probability (Lines 3-7). If all tasks are selected or no task is selected, then the algorithm returns NULL.

Algorithm 4: Pheromone Intensity Method PIM(π_j, b, T_i, p, q).

```

/* Suppose  $\pi_j = (\pi_{j,1}, \pi_{j,2}, \dots, \pi_{j,K_j})$  */
1  $P_i \leftarrow 0$ ;
2  $h_1 \leftarrow -1$ ;
3  $h_2 \leftarrow -1$ ;
4  $i_1 \leftarrow 0$ ;
5  $i_2 \leftarrow n + 1$ ;
6 for  $b_1 = b - 1$  to 1 do
7   if  $\pi_{j,b_1}$  is a 1st stage sub-task then
8      $h_1 \leftarrow 1$ ;
9     Set  $i_1$  to be the task index of  $\pi_{j,b_1}$ ;
10    break;
11   if  $\pi_{j,b_1}$  is a 3rd stage sub-task then
12      $h_1 \leftarrow 3$ ;
13     Set  $i_1$  to be the task index of  $\pi_{j,b_1}$ ;
14    break;
15 for  $b_2 = b$  to  $K_j$  do
16   if  $\pi_{j,b_2}$  is a 1st stage sub-task then
17      $h_2 \leftarrow 1$ ;
18     Set  $i_2$  to be the task index of  $\pi_{j,b_2}$ ;
19    break;
20   if  $\pi_{j,b_2}$  is a 3rd stage sub-task then
21      $h_2 \leftarrow 3$ ;
22     Set  $i_2$  to be the task index of  $\pi_{j,b_2}$ ;
23    break;
24 if  $i_2 = n + 1$  then
25    $P_i \leftarrow +\infty$ ;
26 else if  $h_1 = 1$  and  $h_2 = 1$  then
27    $P_i \leftarrow$ 
28      $(\chi_{i_1,i}^1)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_1,i}^1)^{(1-\lambda_{p,q})} + (\chi_{i_2,i}^2)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_2,i}^2)^{(1-\lambda_{p,q})}$ 
29 else if  $h_1 = 1$  and  $h_2 = 3$  then
30    $P_i \leftarrow$ 
31      $(\chi_{i_1,i}^1)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_1,i}^1)^{(1-\lambda_{p,q})} + (\chi_{i_2,i}^4)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_2,i}^4)^{(1-\lambda_{p,q})}$ 
32 else if  $h_1 = 3$  and  $h_2 = 1$  then
33    $P_i \leftarrow$ 
34      $(\chi_{i_1,i}^3)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_1,i}^3)^{(1-\lambda_{p,q})} + (\chi_{i_2,i}^2)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_2,i}^2)^{(1-\lambda_{p,q})}$ 
35 else if  $h_1 = 3$  and  $h_2 = 3$  then
36    $P_i \leftarrow$ 
37      $(\chi_{i_1,i}^3)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_1,i}^3)^{(1-\lambda_{p,q})} + (\chi_{i_2,i}^4)^{\lambda_{p,q}} \cdot (\hat{\chi}_{i_2,i}^4)^{(1-\lambda_{p,q})}$ 
38 return  $P_i$ ;

```

Algorithm 5: Boundary Update Method BUM(π, B).

```

/* Suppose  $\pi_j = (\pi_{j,1}, \pi_{j,2}, \dots, \pi_{j,K_j})$  */
1  $B' \leftarrow B + 1$ ;
2 while  $\pi_{j,B'}$  is  $T_{swap}$  or a 2nd stage sub-task do
3    $B' \leftarrow B' + 1$ ;
4 if  $B' > K_j + 1$  then
5   return  $-1$ ;
6 return  $B'$ ;

```

- 4) All T_{swap} tasks are removed from π_j since the successive operations may course invalidity of these tasks (Line 8).
- 5) An idle reversed UAV is randomly selected. If there is no idle reserved UAV, an on-demand UAV is introduced (Lines 9-11).
- 6) The sub-tasks of selected tasks are added to $\pi_{j'}$ in the same order as they are sequenced in π_j (Lines 12-15).
- 7) The sub-tasks of selected tasks are removed from π_j (Lines 16-17).

Algorithm 6: Solution Division Method SDM (S^*).

```

1  $S \leftarrow S^*$ ; /* Following operations are performed on
   the copy of input schedule */
2  $U_j \leftarrow \arg \max \{C(\pi_{j,K_j}) | U_j \in \mathbb{U} \cup \mathbb{U}'\}$ ;
3  $\mathbb{T}' \leftarrow \emptyset$ ;
4 for  $\forall a_i \in A_j$  do
5   Put  $T_i$  into  $\mathbb{T}'$  at a 50% probability;
6 if  $|A_j| = |\mathbb{T}'|$  or  $|\mathbb{T}'| = 0$  then
7   return NULL;
8 Remove all  $T_{swap}$  in  $\pi_j$ ;
9 Select an idle UAV  $U_{j'}$ , where reserved one is preferred;
10 if  $U_{j'}$  is an on-demand UAV then
11    $\mathbb{U}' \leftarrow \mathbb{U}' + \{U_{j'}\}$ ;
12  $\pi_{j'} \leftarrow ()$ ; /*  $\pi_{j'}$  is the sub-task sequence of  $U_{j'}$  */
   /* Suppose the current  $\pi_j$  is  $\pi_j = (\pi_{j,1}, \dots, \pi_{j,K_j})$  */
13 for  $k = 1$  to  $K_j$  do
14   if  $\pi_{j,k}$  is the sub-task of a task in  $\mathbb{T}'$  then
15     Add  $\pi_{j,k}$  to the end of  $\pi_{j'}$ ;
16 for  $\forall T_i \in \mathbb{T}'$  do
17   Remove sub-tasks of  $T_i$  from  $\pi_j$ ;
18 Add  $T_{swap}$  tasks to  $\pi_j$  and  $\pi_{j'}$  at appropriate positions by
   Swapping Task Insertion Method;
19 Compute new assignments of tasks in  $\pi_j$  and  $\pi_{j'}$ ;
20 for  $\forall U_{j_1} \in \mathbb{U}$  and  $\forall U_{j_2} \in \mathbb{U}'$  do
21   if  $L^0(U_{j_1}) = L^0(U_{j_2})$  then
22     if  $C(\pi_{j_1,K_{j_1}}) < C(\pi_{j_2,K_{j_2}})$  then
23       Swap the task and UAV assignments of  $U_{j_1}$  and
        $U_{j_2}$ ;
24 Update  $S$  accordingly;
25 return  $S$ ;

```

- 8) T_{swap} tasks are inserted into π_j and $\pi_{j'}$ at appropriate positions (Line 18).
- 9) Based on the new sub-task sequences, new assignments are computed (Line 19).
- 10) Finally, the obtained solution is enhanced by swapping assignments of some UAVs (Lines 20-23).

E. Pheromone Update Method

Pheromone Update Method (PUM) updates pheromone matrices based on the solutions in the Pareto Front. There are three major steps in PUM:

- 1) Solutions in the given Pareto Front are sorted in ascending order of their objectives of total cost and divided equally into $\mathcal{N}$ groups: $PF_1, PF_2, \dots, PF_{\mathcal{N}}$. Apparently, the first group PF_1 consists of solutions with lowest total costs and largest completion times. Conversely, the last group $PF_{\mathcal{N}}$ includes the solutions of highest total costs and smallest completion times.
- 2) Pheromone evaporation operation is performed on every pheromone value of all matrices by multiplying it with the evaporation factor ρ ($0 < \rho < 1$).
- 3) Pheromone Enhancement Method (Algorithm 7) is performed on every colony to increase the pheromone values associated with the given solution group.

Algorithm 7 describes the details of Pheromone Enhancement Method. The pheromone matrices of colony CO_p are enhanced with the solution group PF_p . For each solution, the preference

parameter μ is computed (Line 2). μ is related to the preference of the colony and it depends on the size of PF^* , the colony index and the solution ranking value. Then for each sub-task sequence, the selection pheromone matrices are enhanced (Lines 4-9), followed with the procedure for increasing the pheromone values in insertion pheromone matrices ((Lines 10-47)).

In the procedure of enhancing selection pheromone matrices (Lines 4-9), the task sequence is abstracted from the sub-task sequence (Line 4), which indicates the order that tasks are selected. For every two successive tasks in the task sequence, the corresponding pheromone values from $(\tau_{i,i'})$ and $(\hat{\tau}_{i,i'})$ are enhanced (Lines 5-9). $\Delta\tau$ is defined as an enhancement factor. Both $\Delta\tau$ and μ are used to compute the increment of a pheromone value.

The procedure of enhancing insertion pheromone matrices (Lines 4-9) corresponds to the procedure of employing these matrices in Algorithm 4. Since insertion pheromone matrices are used for inserting 3^{rd} sub-stage tasks, all 3^{rd} stage sub-tasks are checked in the procedure. For each 3^{rd} stage sub-task, the recent transmission sub-tasks before and after it are obtained (Lines 16-33). Finally, the pheromone values are updated accordingly (Lines 34-47). Noted that a 3^{rd} stage sub-task may be the last sub-task except swapping tasks. In this case, only one pair of pheromone matrices would be enhanced (Lines 40-41).

F. Time Complexity of bi-ACO

The best time complexity of bi-ACO is $\Omega(n^3)$ and the worst one is $o(n^4)$. The detailed time complexity analysis for Algorithm 1–7 is given in Appendix A, available online. Apparently, the time complexity of bi-ACO mainly depends on the number of tasks. It is affordable in practise when n is not too big. We evaluate the actual computation time of bi-ACO on the problems of different size (as shown in Table VI) in the next section.

V. COMPUTATIONAL EXPERIMENTS

In the section, the multiple components of bi-ACO framework are calibrated first. Then the calibrated bi-ACO algorithm with the best combination of the components is compared with several baseline algorithms for the similar problems. All compared algorithms are encoded in Java, compiled by Eclipse ide 2022-09 JDK1.8.0 and run on a Alibaba Cloud sever with Intel Xeon (Ice Lake) Platinum 8369B (2.5 GHz) processor and 32 GB of RAM.

A. Performance Metrics

Since the proposed bi-ACO and compared baseline algorithms are all Pareto-based algorithms, and each of which obtains a set of non-dominated solutions, it is difficult to employ single metric to evaluate the overall performance of algorithms. We employ four Pareto-based metrics as follows to measure the obtained non-dominated solutions and evaluate the compared algorithms on aspects of the solution quality, the solution coverage, etc.

Average Quality (AQ) was proposed in [35] to measures the quality of the solution set. A less AQ value indicates the higher

Algorithm 7: Pheromone Enhancement Method PEM (CO_p).

```

1 for  $\forall S = \langle A, \Pi \rangle \in PF_p$  do
  /* Suppose  $S$  is ranked as the  $r^{th}$  solution in  $PF_p$  */
2   $\mu \leftarrow \frac{p-1}{N} + \frac{r}{|PF^*|}$ ;
3  for  $\forall \pi_j \in \Pi$  do
    /* Suppose there are  $N$  tasks assigned to  $U_j$  */
4    Obtain task sequence  $T_{i_1}, T_{i_2}, \dots, T_{i_N}$  from  $\pi_j$ 
    satisfying  $S(T_{i_1,1}) < S(T_{i_2,1}) < \dots < S(T_{i_N,1})$ ;
5     $i \leftarrow 0$ ;
    /* Increase pheromone values in selection pheromone matrices */
6    for  $i' = i_1, \dots, i_N$  do
7       $\tau_{i,i'} \leftarrow \tau_{i,i'} + \Delta\tau \times \mu$ ;
8       $\hat{\tau}_{i,i'} \leftarrow \hat{\tau}_{i,i'} + \Delta\tau \times (1 - \mu)$ ;
9       $i \leftarrow i'$ ;
    /* Suppose  $\pi_j = (\pi_{j,1}, \pi_{j,2}, \dots, \pi_{j,K_j})$  */
10   for  $k = 1, \dots, K_j$  do
11     if  $\pi_{j,k}$  is not a  $3^{rd}$  stage sub-task then
12       continue;
13     Set  $i$  to be the task index of  $\pi_{j,k}$ ;
14      $i_1 \leftarrow 0$ ;  $i_2 \leftarrow n + 1$ ;
15      $h_1 \leftarrow -1$ ;  $h_2 \leftarrow -1$ ;
16     for  $k_1 = k - 1$  to 1 do
17       if  $\pi_{j,k_1}$  is a  $1^{st}$  stage sub-task then
18          $h_1 \leftarrow 1$ ;
19         Set  $i_1$  to be the task index of  $\pi_{j,k_1}$ ;
20         break;
21       if  $\pi_{j,k_1}$  is a  $3^{st}$  stage sub-task then
22          $h_1 \leftarrow 3$ ;
23         Set  $i_1$  to be the task index of  $\pi_{j,k_1}$ ;
24         break;
25     for  $k_2 = k + 1$  to  $K_j$  do
26       if  $\pi_{j,k_2}$  is a  $1^{st}$  stage sub-task then
27          $h_2 \leftarrow 1$ ;
28         Set  $i_2$  to be the task index of  $\pi_{j,k_2}$ ;
29         break;
30       if  $\pi_{j,k_2}$  is a  $3^{st}$  stage sub-task then
31          $h_2 \leftarrow 3$ ;
32         Set  $i_2$  to be the task index of  $\pi_{j,k_2}$ ;
33         break;
    /* Increase pheromone values in insertion pheromone matrices */
34   if  $h_1 = 1$  then
35      $\chi_{i_1,i}^1 \leftarrow \chi_{i_1,i}^1 + \Delta\tau \times \mu$ ;
36      $\hat{\chi}_{i_1,i}^1 \leftarrow \hat{\chi}_{i_1,i}^1 + \Delta\tau \times (1 - \mu)$ ;
37   else if  $h_1 = 3$  then
38      $\chi_{i_1,i}^3 \leftarrow \chi_{i_1,i}^3 + \Delta\tau \times \mu$ ;
39      $\hat{\chi}_{i_1,i}^3 \leftarrow \hat{\chi}_{i_1,i}^3 + \Delta\tau \times (1 - \mu)$ ;
40   if  $i_2 = n + 1$  then
41     continue;
42   if  $h_2 = 1$  then
43      $\chi_{i,i_2}^2 \leftarrow \chi_{i,i_2}^2 + \Delta\tau \times \mu$ ;
44      $\hat{\chi}_{i,i_2}^2 \leftarrow \hat{\chi}_{i,i_2}^2 + \Delta\tau \times (1 - \mu)$ ;
45   else if  $h_2 = 3$  then
46      $\chi_{i,i_2}^4 \leftarrow \chi_{i,i_2}^4 + \Delta\tau \times \mu$ ;
47      $\hat{\chi}_{i,i_2}^4 \leftarrow \hat{\chi}_{i,i_2}^4 + \Delta\tau \times (1 - \mu)$ ;

```

average solution quality for a given set of non-dominated solutions. The metric is defined as (24)–(26). f_j is the normalized j^{th} objective of the solution S and n is the number of the objectives ($n = 2$ in the paper). Min-Max normalization is employed to normalize the objectives.

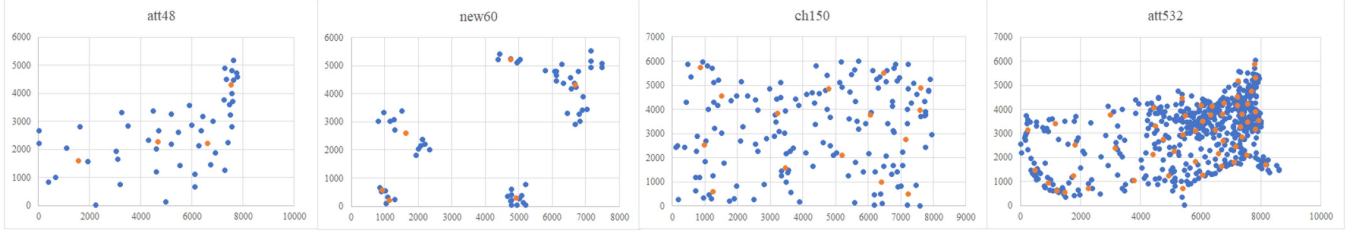


Fig. 5. Maps.

Maximum Spread (MS) was defined in [36] to measure how well PF^* is covered by the given non-dominated solution set PF . PF^* is the near-optimal Pareto front based on the union of the non-dominated solutions obtained by all compared algorithms. The larger the metric is, the more the near-optimal Pareto front PF^* is covered by the given PF . The metric is formulated as (27).

Maximum Distance ($D_{\max}$) and average Distance (D_{avg}) were defined in [37] to measure how close PF is to PF^* . Smaller D_{avg} and $D_{\max}$ values correspond to a better approximation to PF^* . The metrics are defined as (28) and (29), respectively. $\Delta(S, S^*)$ is the relative distance between two solutions (30).

$$AQ(PF) = \sum_{\lambda \in \Lambda} \frac{s_a(PF, \lambda)}{|\Lambda|} \quad (24)$$

$$s_a(PF, \lambda) = \min_{S \in PF} \left\{ \max_j \{ \lambda_j f_j(S) \} + \sum_j \frac{\lambda_j f_j(S)}{100} \right\} \quad (25)$$

$$\Lambda = \{(\lambda_1, \dots, \lambda_n) \mid$$

$$\lambda_j \in \left\{ 0, \frac{1}{50}, \frac{2}{50}, \dots, \frac{49}{50}, 1 \right\}, \sum_{j=1}^n \lambda_j = 1 \} \quad (26)$$

$$MS(PF) = \sqrt{\frac{1}{n} \sum_{j=1}^n \left(\frac{\max_{S \in PF} f_j(S) - \min_{S \in PF} f_j(S)}{\max_{S \in PF^*} f_j(S) - \min_{S \in PF^*} f_j(S)} \right)} \quad (27)$$

$$D_{\max} = \max_{S^* \in PF^*} \{ \min_{S \in PF} \Delta(S, S^*) \} \quad (28)$$

$$D_{avg} = \sum_{S^* \in PF^*} \frac{\min_{S \in PF} \Delta(S, S^*)}{|PF^*|} \quad (29)$$

$$\Delta(S, S^*) = \max_{j=1}^n \left\{ \frac{f_j(S) - f_j(S^*)}{\max_{S \in PF^*} f_j(S) - \min_{S \in PF^*} f_j(S)} \right\} \quad (30)$$

B. Testing Instances

The performance of compared algorithms are tested on the testing instances from the literatures [33], [34], [38]. We employ 4 maps att48, new60, ch150, and att532 to simulate the SD distribution, where att48, ch150 and att532 are benchmark maps from TSPLIB [38]. Map new60 is a randomly generated map, where a given number of base stations are distributed

randomly first and then the SDs are distributed randomly around each base station. The nodes in these maps are taken as the tasks in the considered problems. The tasks in these maps are depicted as blue nodes in Fig. 5. The numbers of tasks in map att48, new60, ch150 and att532 are 48, 60, 150 and 532, respectively. The size of these maps are $6000 \text{ m} \times 8000 \text{ m}$, $6000 \text{ m} \times 8000 \text{ m}$, $6000 \text{ m} \times 8000 \text{ m}$ and $7000 \text{ m} \times 8000 \text{ m}$, respectively. The number of base stations $|\mathbb{B}|$ is set to $\lfloor \frac{n}{10} \rfloor$. The base stations are distributed as orange nodes in Fig. 5.

For the UAV settings, the number of reserved UAVs m is set to $\lceil \frac{n}{20} \rceil$ or $\lceil \frac{n}{25} \rceil$. Reserved UAVs are randomly distributed on base stations. The flying speed of UAVs ν is set to 20, 30, 40 or 50 m/s . The computation time $P(T_i)$ (in second) follows the uniform distribution $U(10, 100)$. The data transmission time $DT(T_i)$ (in second) is set to be linearly related to $P(T_i)$, i.e., $DT(T_i) = \theta_{dt} \times P(T_i)$, where $\theta_{dt} \in \{0.25, 0.5, 0.75\}$. The result transmission time $RT(T_i)$ (in second) is set to be $P(T_i) \times 0.01$. The deadline $D(T_i)$ is set to $CT(T_i)_{\min} \times \theta_d$, where $\theta_d \in \{5, 10, 15\}$. $CT(T_i)_{\min}$ is defined as the earliest completion time of T_i , which is computed by assuming the closest UAV to process the task in an exclusive manner. The unit price setting of UAVs satisfies the equation $\alpha = \theta_p \times \beta$ with $\theta_p \in \{2, 4, 6\}$.

There are $4 \times 2 \times 4 \times 3 \times 3 \times 3 = 864$ instance combinations ($n \in \{48, 76, 150, 532\}$, $m \in \{\lceil \frac{n}{20} \rceil, \lceil \frac{n}{25} \rceil\}$, $\nu \in \{20, 30, 40, 50\}$, $\theta_{dt} \in \{0.25, 0.5, 0.75\}$, $\theta_d \in \{5, 10, 15\}$, $\theta_p \in \{2, 4, 6\}$).

C. Parameter and Component Calibration

In order to show the advancement of the component SDM, two variants of the main framework are calibrated: the one with SDM (bi-ACO), and the one without it (bi-ACO-noSDM). In FSGM, each ant breaks the loop at a given probability which is computed based on the index of the colony and the breaking factor θ_b (Line 25, Algorithm 2). To demonstrate the usefulness of this loop-breaking step, the variants of FSGM with different settings of $\theta_b \in \{0, 0.1, 0.3, 0.5, 0.7, 0.9\}$ are calibrated. FSGM with $\theta_b = 0$ is the variant without the breaking step. Besides, one algorithm parameter is calibrated: the colony number $\mathcal{N} \in \{1, 5, 10, 20\}$. There are $2 \times 6 \times 4 = 48$ variants of the proposed bi-ACO in total to calibrate. Three random instances are generated for each possible combination of testing instances. Therefore, the variants of the proposal are calibrated on $864 \times 3 = 2592$ instances in total. In other words, $48 \times 2592 = 124,416$ experimental results are obtained. These

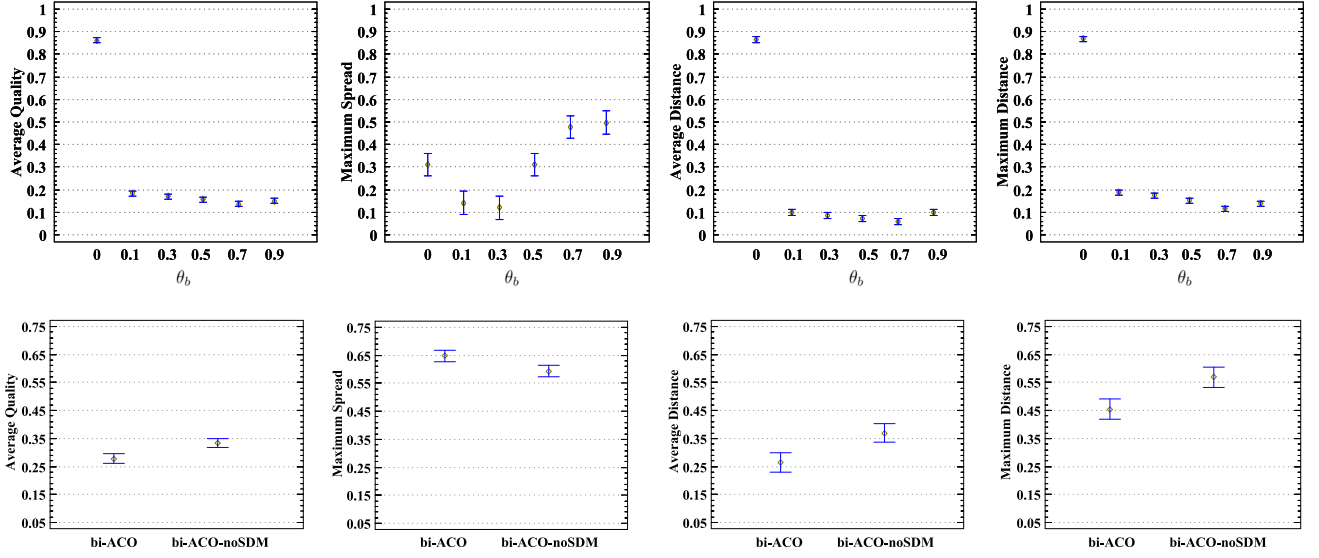


Fig. 6. Mean plots of variants across all instances with 95.0% Tukey HSD intervals.

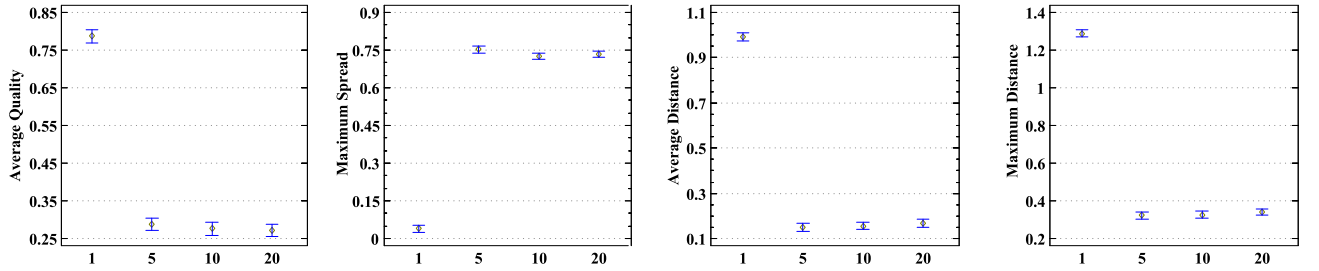


Fig. 7. Mean plots of variants with different $\mathcal{N}$ across all instances with 95.0% Tukey HSD intervals.

experimental results are analyzed by the multi-factor analysis of variance (ANOVA) technique.

Fig. 6 shows four Pareto-based metrics for variants with 95.0% Tukey Honest Significant Difference (HSD) intervals. It can be observed that bi-ACO with SDM outperforms the one without SDM on all metrics. The reason lies in that SDM can improve the diversity of solutions in the non-dominated solution set. Fig. 6 also shows a statistically significant difference for the performance of FSGM variants on all metrics. FSGM variants with the loop-breaking step have better performance than the one without it ($\theta_b = 0$). We can observe that FSGM with $\theta_b = 0.7$ has the best performance on the most metrics. If there is no loop-breaking step, every UAV is assigned as many tasks as possible, which would lead to longer completion time. Breaking the loop at a given probability can balance the workload of UAVs to some extent. It is reasonable that FSGM with the loop-breaking step demonstrates better performance.

As shown in Fig. 7, $\mathcal{N} = 1$ has the worst performance. The performance of variants with the other settings ($\mathcal{N} = 5, 10, 20$) are not significantly different. In the variant with $\mathcal{N} = 5$, less ants are introduced, which means less computation time. Therefore, it is reasonable to set $\mathcal{N} = 5$ in the proposal.

Therefore, the best parameter and component combination is bi-ACO with SDM and loop-breaking components, and $\mathcal{N} = 5$.

D. Algorithm Comparison

The proposed bi-ACO is compared with five baseline algorithms for the similar problems: 1) NSGA-II [33], 2) Simulated Annealing (SA) [39], 3) Variable Neighborhood Descent (VND) [40], 4) Greedy Local Search (GLS) [33] and 5) ACO-DSP [16]. We adopt the similar component and parameter settings given in [33] for NSGA-II, SA, VND and GLS. In [16], ACO-DSP is an ACO variant for drone scheduling problem. ACO-DSP is developed for single objective. In order to solve the bi-objective problem under study, we modify ACO-DSP in [16] by integrating it into an iterative heuristics with different preferences of objectives.

To compare the algorithms, 5 instances are randomly generated for each instance combination, i.e., $864 \times 6 = 5,184$ new instances for comparison. Four Pareto-based metrics for the compared algorithms are shown in Table V–VI. Table V illustrates that bi-ACO has the best performance on metric AQ (0.189) for all instance combinations. ACO-DSP is the second best one on the metric AQ (0.460), while VND is the worst one on the metric (0.729). For the metric MS, SA (0.958) outperforms all the other algorithms on most cases. Although bi-ACO does not have the best performance on metric MS, it is the second best one (0.940). VND performs the worst (0.016). Table VI shows

TABLE V
EXPERIMENTAL RESULTS ON METRICS AQ AND MS

Param.	Value	Average Quality (AQ)						Maximum Spread (MS)					
		bi-ACO	GLS	NSGA-II	SA	VND	ACO-DSP	bi-ACO	GLS	NSGA-II	SA	VND	ACO-DSP
n	48	0.141	0.731	0.386	0.453	0.742	0.490	0.969	0.100	0.393	0.920	0.023	0.708
	60	0.131	0.799	0.527	0.606	0.801	0.342	0.969	0.075	0.355	0.955	0.011	0.715
	150	0.208	0.660	0.496	0.571	0.665	0.554	0.821	0.040	0.223	0.990	0.010	0.657
	532	0.275	0.697	0.648	0.713	0.706	0.453	1.000	0.073	0.241	0.969	0.019	0.737
ν	20	0.218	0.708	0.446	0.562	0.705	0.503	0.910	0.073	0.339	0.963	0.017	0.599
	30	0.193	0.717	0.493	0.582	0.727	0.465	0.940	0.077	0.294	0.946	0.014	0.706
	40	0.177	0.727	0.542	0.594	0.732	0.446	0.949	0.076	0.294	0.972	0.013	0.737
	50	0.167	0.736	0.576	0.607	0.751	0.425	0.960	0.063	0.286	0.952	0.020	0.775
m	$\lceil n/20 \rceil$	0.187	0.734	0.512	0.577	0.743	0.436	0.943	0.071	0.295	0.968	0.012	0.738
	$\lceil n/25 \rceil$	0.190	0.710	0.517	0.595	0.715	0.484	0.936	0.073	0.312	0.949	0.019	0.671
θ_{dt}	0.25	0.181	0.736	0.509	0.596	0.734	0.483	0.944	0.084	0.448	0.956	0.014	0.686
	0.5	0.181	0.785	0.561	0.633	0.786	0.447	0.942	0.064	0.291	0.981	0.017	0.707
	0.75	0.204	0.646	0.473	0.530	0.666	0.449	0.934	0.068	0.170	0.937	0.017	0.721
θ_d	5	0.145	0.614	0.507	0.615	0.606	0.320	0.992	0.051	0.429	0.875	0.022	0.932
	10	0.234	0.741	0.514	0.584	0.757	0.618	0.844	0.110	0.302	1.000	0.020	0.429
	15	0.187	0.811	0.522	0.559	0.823	0.442	0.983	0.055	0.178	1.000	0.005	0.753
θ_p	2	0.178	0.811	0.595	0.729	0.813	0.504	0.934	0.085	0.419	0.967	0.011	0.662
	4	0.196	0.687	0.477	0.535	0.695	0.449	0.944	0.064	0.265	0.958	0.017	0.714
	6	0.193	0.668	0.470	0.494	0.677	0.427	0.941	0.067	0.225	0.950	0.020	0.737
Average		0.189	0.722	0.514	0.586	0.729	0.460	0.940	0.072	0.303	0.958	0.016	0.704

TABLE VI
EXPERIMENTAL RESULTS ON METRICS D_{avg} AND D_{max}

Param.	Value	Average Distance (D_{avg})						Maximum Distance (D_{max})					
		bi-ACO	GLS	NSGA-II	SA	VND	ACO-DSP	bi-ACO	GLS	NSGA-II	SA	VND	ACO-DSP
n	48	0.012	0.813	0.412	0.566	0.823	0.625	0.063	0.932	0.679	0.714	0.931	0.828
	60	0.004	0.857	0.612	0.772	0.861	0.369	0.035	0.938	0.834	0.881	0.935	0.655
	150	0.047	0.774	0.546	0.705	0.783	0.715	0.227	0.936	0.853	0.897	0.932	0.837
	532	0.097	0.783	0.702	0.826	0.799	0.529	0.365	0.960	0.931	0.966	0.963	0.642
ν	20	0.080	0.787	0.465	0.683	0.788	0.638	0.265	0.932	0.755	0.865	0.926	0.838
	30	0.037	0.807	0.549	0.716	0.819	0.583	0.182	0.941	0.819	0.865	0.940	0.767
	40	0.026	0.808	0.609	0.727	0.816	0.529	0.138	0.941	0.855	0.864	0.942	0.700
	50	0.016	0.825	0.650	0.742	0.842	0.488	0.104	0.952	0.869	0.865	0.954	0.657
m	$\lceil n/20 \rceil$	0.038	0.818	0.571	0.711	0.830	0.539	0.166	0.938	0.817	0.851	0.941	0.728
	$\lceil n/25 \rceil$	0.041	0.796	0.565	0.723	0.803	0.581	0.179	0.945	0.832	0.879	0.940	0.754
θ_{dt}	0.25	0.048	0.762	0.540	0.709	0.764	0.582	0.151	0.885	0.753	0.855	0.879	0.741
	0.5	0.034	0.897	0.641	0.789	0.898	0.556	0.140	0.982	0.884	0.903	0.983	0.715
	0.75	0.037	0.761	0.524	0.654	0.788	0.541	0.226	0.957	0.837	0.836	0.960	0.766
θ_d	5	0.011	0.609	0.464	0.627	0.606	0.398	0.039	0.836	0.716	0.834	0.827	0.554
	10	0.077	0.870	0.616	0.769	0.893	0.763	0.320	0.990	0.874	0.894	0.995	0.920
	15	0.031	0.941	0.624	0.756	0.950	0.518	0.159	0.999	0.883	0.867	0.999	0.747
θ_p	2	0.038	0.875	0.650	0.854	0.877	0.639	0.157	0.966	0.853	0.946	0.966	0.798
	4	0.040	0.782	0.535	0.672	0.796	0.535	0.184	0.937	0.811	0.835	0.936	0.729
	6	0.041	0.763	0.520	0.625	0.776	0.505	0.176	0.921	0.809	0.814	0.920	0.694
Average		0.040	0.807	0.568	0.717	0.816	0.560	0.172	0.942	0.824	0.865	0.940	0.741

that bi-ACO has the best performance on both D_{avg} (0.04) and D_{max} (0.172) for all instance combinations. ACO-DSP is the second best one on metrics D_{avg} (0.56) and D_{max} (0.741). VND and GLS have the worst performance on D_{avg} (0.816) and D_{max} (0.942), respectively. Table VII shows that the computation time

of bi-ACO is more time-consuming than that of GLS and SA but less than that of VND, NSGA-II and ACO-DSP for all cases. According to the performance of compared algorithms on all metrics, we can conclude that bi-ACO is the most effective algorithm among all compared ones.

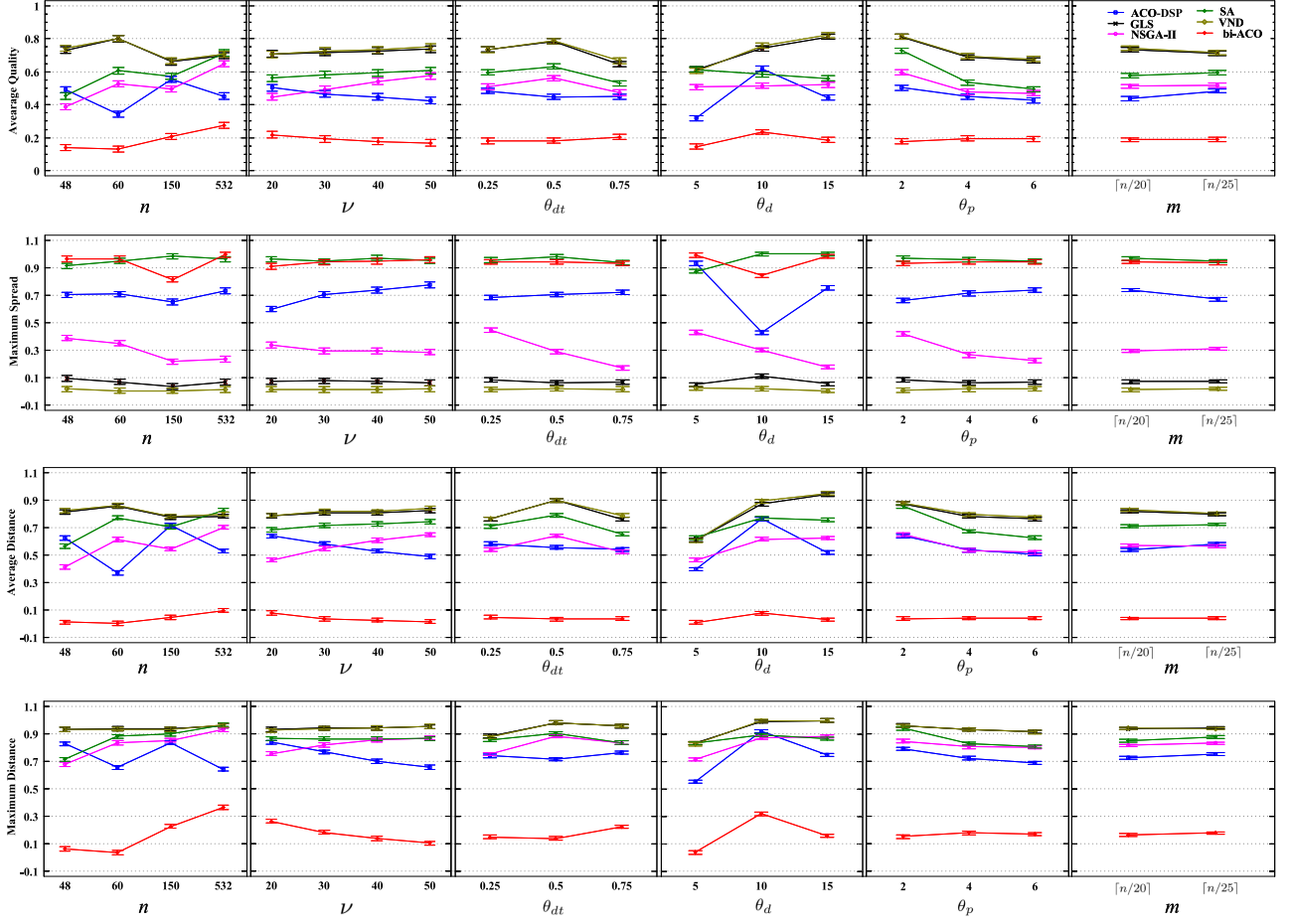


Fig. 8. Interactions between instance parameters and the compared algorithms with 95.0% Tukey HSD intervals.

TABLE VII
COMPUTATION TIME OF COMPARED ALGORITHMS

n, θ_d	Time (s)					
	bi-ACO	GLS	NSGA-II	SA	VND	ACO-DSP
48,5	12.21	2.87	51.82	2.29	114.11	185.74
60,5	18.42	4.00	131.32	5.30	190.30	283.31
150,5	81.17	3.94	462.26	7.74	685.05	914.08
532,5	637.13	2.17	1883.86	16.34	1782.55	9970.69
48,10	11.68	2.87	86.62	1.58	282.66	193.08
60,10	17.95	4.49	315.15	3.75	352.01	297.58
150,10	79.99	4.98	1390.98	5.99	1076.64	915.22
532,10	723.25	4.28	5502.38	11.85	1959.62	9990.06
48,15	12.03	5.37	92.27	1.27	195.42	186.93
60,15	18.33	4.62	257.21	2.70	772.10	293.46
150,15	79.01	4.81	1586.62	5.70	960.77	912.98
532,15	647.02	5.32	10263.7	10.77	906.06	9909.66
Avg.	194.85	4.15	1835.36	6.28	773.11	2837.73

In order to show the robustness and reliability of the proposed algorithm, we conduct the parameter sensitivity experiments. Fig. 8 depicts the interactions between instance parameters and the compared algorithms. From Fig. 8 we can observe that bi-ACO is less sensitive to the parameter changes for most cases. For the metric of AQ, bi-ACO is a little sensitive to

the parameter n . For the metric of MS, bi-ACO is robust on most parameters except n and θ_d . For the metric of D_{avg} , bi-ACO is reliable on all parameters. For the metric of D_{max} , bi-ACO is sensitive only on parameters n and θ_d . Besides, we design three kinds of maps MAP1, MAP2 and MAP3 to show the performances of the compared algorithms on the different SD distributions. MAP1 includes 6 base stations and 60 SDs which are randomly generated obeying uniform distribution. In MAP2, 6 base stations are randomly generated obeying uniform distribution first, and then 60 SDs are uniformly distributed within 1000 meters of base stations. In MAP3, 6 base stations are randomly generated obeying uniform distribution first, and then 60 SDs are normally distributed around base stations. Fig. 9 depicts the performance of the compared algorithms with different SD distributions. It can be seen from the figure that the proposed algorithm is relatively stable with different SD distributions compared to other algorithms. Therefore, we can conclude that bi-ACO is robust and reliable for the problem under study.

Based on the experimental results, we can conclude that the proposed bi-ACO is the most effective and robust algorithm for the problem under study. It consumes more time than some of compared algorithms, however, it is acceptable for practical applications with respect to the solution quality of bi-ACO.

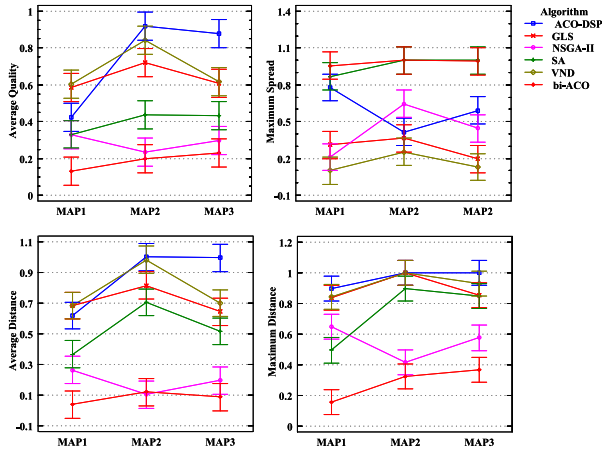


Fig. 9. Interactions between SD distributions and the compared algorithms with 95.0% Tukey HSD intervals.

In order to show the feasibility of the proposed bi-ACO in practise, we perform the field tests. We adopt the remote control method for the field tests, i.e., the proposed bi-ACO runs on a powerful server and the trajectory plans it produces are sent to UAVs to perform. The field tests show that it is feasible to implement the proposed algorithm on real UAVs. The detail of the field tests is given in Appendix B, available online.

VI. CONCLUSION

In this paper, we studied the problem jointly considering the multi-UAV trajectory planning and multi-stage task offloading. The problem involves the energy, deadline and precedence constraints, and the objectives are to minimize the total cost and the completion time. To solve the problem, we proposed the bi-ACO algorithm to obtain a non-dominated set containing promising solutions. In bi-ACO framework, massive ants are divided into several heterogeneous colonies, and ants in different colonies has different preferences of objectives to explore. Each colony maintains five pairs of pheromone matrices for computing the probability for selecting tasks and determining the insertion positions of tasks. Each ant first adopts component FSGM to generate a feasible solution according to the pheromone matrices and heuristic values, and then calls SDM to further improve the obtained solution. At the end of each iteration of bi-ACO, PUM is used to update pheromone matrices based on the obtained non-dominated solutions. Algorithm calibration was conducted to determine the best component and parameter combination for the bi-ACO framework. Then we evaluate performance of bi-ACO, NSGA-II, SA, VND, GLS and ACO by four Pareto-based metrics. The experimental results illustrates that bi-ACO outperforms the compared algorithms on most metrics.

REFERENCES

- [1] J. Wang, C. Jin, Q. Tang, N. N. Xiong, and G. Srivastava, "Intelligent ubiquitous network accessibility for wireless-powered MEC in UAV-assisted B5G," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 2801–2813, Fourth Quarter, 2021.
- [2] K. Li, W. Ni, X. Wang, R. P. Liu, S. S. Kanhere, and S. Jha, "Energy-efficient cooperative relaying for unmanned aerial vehicles," *IEEE Trans. Mobile Comput.*, vol. 15, no. 6, pp. 1377–1386, Jun. 2016.
- [3] N. Zhao et al., "Caching UAV assisted secure transmission in hyper-dense networks based on interference alignment," *IEEE Trans. Commun.*, vol. 66, no. 5, pp. 2281–2294, May 2018.
- [4] P. Valianti, P. Kolios, and G. Ellinas, "Energy-aware tracking and jamming rogue UAVs using a swarm of pursuer UAV agents," *IEEE Syst. J.*, vol. 17, no. 1, pp. 1524–1535, Mar. 2023.
- [5] J. Wang, K. Liu, and J. Pan, "Online UAV-mounted edge server dispatching for mobile-to-mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1375–1386, Feb. 2020.
- [6] S. Hu, X. Yuan, W. Ni, X. Wang, E. Hossain, and H. V. Poor, "OFDMA-F2L: Federated learning with flexible aggregation over an OFDMA air interface," *IEEE Trans. Wireless Commun.*, early access, Jan. 15, 2024, doi: 10.1109/TWC.2023.3334691.
- [7] H. Liu et al., "An iterative two-phase optimization method based on divide and conquer framework for integrated scheduling of multiple UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 9, pp. 5926–5938, Sep. 2021.
- [8] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14198–14211, Dec. 2020.
- [9] H. Huang, C. Hu, J. Zhu, M. Wu, and R. Malekian, "Stochastic task scheduling in UAV-based intelligent on-demand meal delivery system," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 13040–13054, Aug. 2022.
- [10] Q. Luan, H. Cui, L. Zhang, and Z. Lv, "A hierarchical hybrid subtask scheduling algorithm in UAV-assisted MEC emergency network," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12737–12753, Jul. 2022.
- [11] Y. Du, K. Yang, K. Wang, G. Zhang, Y. Zhao, and D. Chen, "Joint resources and workflow scheduling in UAV-enabled wirelessly powered MEC for IoT systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10187–10200, Oct. 2019.
- [12] K. Zhang, X. Gui, D. Ren, and D. Li, "Energy latency tradeoff for computation offloading in UAV-assisted multiaccess edge computing system," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6709–6719, Apr. 2021.
- [13] Y. Liao, X. Chen, S. Xia, Q. Ai and Q. Liu, "Energy minimization for UAV swarm-enabled wireless inland ship MEC network with time windows," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 594–608, Jun. 2023.
- [14] Z. Wang, R. Liu, Q. Liu, J. S. Thompson, and M. Kadoch, "Energy-efficient data collection and device positioning in UAV-assisted IoT," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1122–1139, Feb. 2020.
- [15] N. Bartolini, A. Coletta, G. Maselli, and A. Khalifeh, "A multi-trip task assignment for early target inspection in squads of aerial drones," *IEEE Trans. Mobile Comput.*, vol. 20, no. 11, pp. 3099–3116, Nov. 2021.
- [16] Z.-H. Sun, X. Luo, E. Q. Wu, T.-Y. Zuo, Z.-R. Tang, and Z. Zhuang, "Monitoring scheduling of drones for emission control areas: An ant colony-based approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11699–11709, Aug. 2022.
- [17] E. S. Rigas, P. Kolios, M. Mavrouniotis, and G. Ellinas, "Scheduling a fleet of drones for monitoring missions with spatial, temporal, and energy constraints," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15133–15145, Sep. 2022.
- [18] S. Hu, X. Yuan, W. Ni, X. Wang and A. Jamalipour, "RIS-assisted jamming rejection and path planning for UAV-borne IoT platform: A new deep reinforcement learning framework," *IEEE Internet Things J.*, vol. 10, no. 22, pp. 20162–20173, Nov. 2023.
- [19] M. Kang and S. W. Jeon, "Energy-efficient data aggregation and collection for Multi-UAV-Enabled IoT networks," *IEEE Wireless Commun. Lett.*, vol. 13, no. 4, pp. 1004–1008, Apr. 2024.
- [20] N. Zhao et al., "Joint trajectory and precoding optimization for UAV-Assisted NOMA networks," *IEEE Trans. Commun.*, vol. 67, no. 5, pp. 3723–3735, May 2019.
- [21] X. Pang, N. Zhao, J. Tang, C. Wu, D. Niyato and K. Wong, "IRS-Assisted secure UAV transmission via joint trajectory and beamforming design," *IEEE Trans. Commun.*, vol. 70, no. 2, pp. 1140–1152, Feb. 2022.
- [22] N. L. Prasad and B. Ramkumar, "3-D deployment and trajectory planning for relay based UAV assisted cooperative communication for emergency scenarios using Dijkstra's algorithm," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5049–5063, Apr. 2023.
- [23] E. Eldeeb, J. M. d.S. Sant'Ana, D. E. Pérez, M. Shehab, N. H. Mahmood, and H. Alves, "Multi-UAV path learning for age and power optimization in IoT with UAV battery recharge," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5356–5360, Apr. 2023.
- [24] C. Sun, W. Ni, and X. Wang, "Joint computation offloading and trajectory planning for UAV-assisted edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5343–5358, Aug. 2021.

- [25] C. Lin et al., "Near optimal charging schedule for 3-D wireless rechargeable sensor networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 5343–5358, Jun. 2023.
- [26] Y. Miao, K. Hwang, D. Wu, Y. Hao, and M. Chen, "Drone swarm path planning for mobile edge computing in Industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 19, no. 5, pp. 6836–6848, May 2023.
- [27] A. Khochare, F. B. Sorbelli, Y. W. Simmhan, and S. K. Das, "Improved algorithms for co-scheduling of edge analytics and routes for UAV fleet missions," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 17–33, Feb. 2024. doi: [10.1109/TNET.2023.3277810](https://doi.org/10.1109/TNET.2023.3277810).
- [28] J. Tian, D. Wang, H. Zhang, and D. Wu, "Service satisfaction-oriented task offloading and UAV scheduling in UAV-enabled MEC networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8949–8964, Dec. 2023. doi: [10.1109/TWC.2023.3267330](https://doi.org/10.1109/TWC.2023.3267330).
- [29] L. Yu, X. Sun, S. Shao, Y. Chen, and R. Albelaihi, "Backhaul-aware drone base station placement and resource management for FSO-based drone-assisted mobile networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1659–1668, May/Jun. 2023.
- [30] J. Luo, J. Song, F. C. Zheng, L. Gao, and T. Wang, "User-centric UAV deployment and content placement in cache-enabled multi-UAV networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5656–5660, May 2022.
- [31] Y. Liu, W. Huangfu, H. Zhou, H. Zhang, J. Liu and K. Long, "Fair and energy-efficient coverage optimization for UAV placement problem in the cellular network," *IEEE Trans. Commun.*, vol. 70, no. 6, pp. 4222–4235, Jun. 2022.
- [32] F. Fazel, J. Abouei, M. Jaseemuddin, A. Anpalagan and K. N. Plataniotis, "Secure throughput optimization for cache-enabled multi-UAVs networks," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7783–7801, May 2022.
- [33] J. Zhu, X. Wang, H. Huang, S. Cheng, and M. Wu, "A NSGA-II algorithm for task scheduling in UAV-enabled MEC system," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9414–9429, Jul. 2022.
- [34] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [35] A. Jaskiewicz, "Do multiple-objective metaheuristics deliver on their promises? A computational experiment on the set-covering problem," *IEEE Trans. Evol. Comput.*, vol. 7, no. 2, pp. 133–143, Apr. 2003.
- [36] K. C. Tan, C. K. Goh, Y. J. Yang, and T. H. Lee, "Evolving better population distribution and exploration in evolutionary multi-objective optimization," *Eur. J. Oper. Res.*, vol. 171, no. 2, pp. 463–495, 2006.
- [37] P. Czyzszak, and A. Jaskiewicz, "Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization," *J. Multi Criteria Decis. Anal.*, vol. 7, no. 1, pp. 34–47, 1998.
- [38] G. Reinelt, "TSPLIB—a traveling salesman problem library," *INFORMS J. Comput.*, vol. 3, no. 3, pp. 376–384, 1991.
- [39] H. Daryanavard and A. Harifi, "UAV path planning for data gathering of IoT nodes: Ant colony or simulated annealing optimization," in *Proc. 3rd Int. Conf. Internet Things Appl.*, 2019, pp. 1–4.
- [40] H. Liu et al., "An iterative two-phase optimization method based on divide and conquer framework for integrated scheduling of multiple UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 9, pp. 5926–5938, Sep. 2021.



Yiqian Wang received the B.E degree in computer science and technology from the Nanjing University of Posts and Telecommunications, Nanjing, China. He is working toward the master degree in the the School of Computer Science and Engineering, Southeast University, Nanjing. His research interests include trajectory planning and task scheduling.



Jie Zhu received the PhD degree in applied computer science from the School of Computer Science and Engineering, Southeast University, Nanjing, in 2011. From 2008 to 2009, she was with the Department of Electrical and Computer Engineering, University of Western Ontario, London, ON, Canada, as a visiting student. She joined the Nanjing University of Post and Telecommunication in 2014, where she is currently an associate professor with the School of Computer Science. Her research interests include machine scheduling, project scheduling, workflow optimization, and

cloud computing, among which task scheduling and resource provisioning in clouds are her current core research areas.



Haiping Huang (Member, IEEE) received the BEng and MEng degrees in computer science and technology from the Nanjing University of Posts & Telecommunications, Nanjing, China, in 2002 and 2005, respectively, and the PhD degree in computer application technology from Soochow University, China, in 2009. From May 2013 to November 2013, he was a visiting scholar with the School of Electronics and Computer Science, University of Southampton, Southampton, U.K. He is currently a professor with the School of Computer Science and Technology,

Nanjing University of Posts and Telecommunications. His research interests include information security and privacy protection of wireless sensor networks.



Fu Xiao (Member, IEEE) received the PhD degree in computer science and technology from the Nanjing University of Science and Technology, Nanjing, China, in 2007. He is currently a professor and a PhD supervisor with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing. His research interests include wireless sensor networks.